DLS

LECTURE · GENERATIVE MODELS

Diffusion Distillation

From 50+ sampling steps to 1–4: how a slow teacher becomes a fast student

MOTIVATION

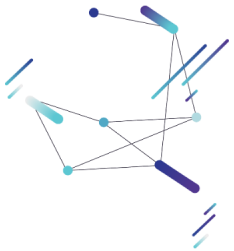
Sampling is the bottleneck

40–100

network calls (NFEs) per sample for a typical teacher (often ×2 with classifier-free guidance)

1–4

NFEs for a distilled student, guidance baked in: 10–50× fewer calls

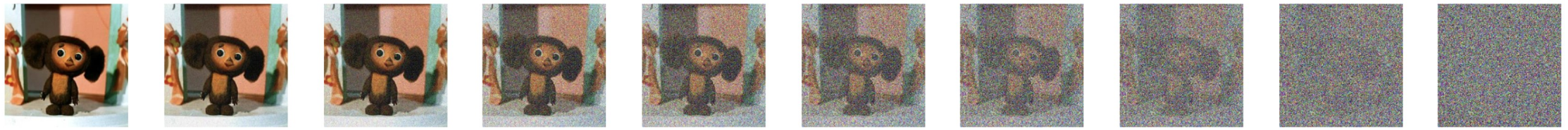


THE PROBLEM

Slow inference: the backward process

Forward process (прямой процесс)

$$q(x_t \mid x_{t-1})$$



$T = 0$

$x_0 \sim P_{data}$

Backward process (обратный процесс)

$$p(x_{t-1} \mid x_t)$$

$T = 1000$

$\epsilon \sim \mathcal{N}(0, 1)$

Training is cheap

One random t per example: noise x_0, predict ε. No simulation.

Sampling is not

Must walk back from T = 1000 to 0: every step is a full network call.

Even with good solvers

DDIM / DPM-Solver still need ~25–50 steps for top quality.

RECAP

Diffusion and classifier-free guidance

Diffusion in one box

forward

$x_t = \alpha_t \cdot x_0 + \sigma_t \cdot \varepsilon,\ \varepsilon \sim N(0, I)$

train (ε-, v- or x_0-prediction)

$\min E \|\varepsilon_\theta(x_t, t, c) - \varepsilon\|^2$

sample: probability-flow ODE

$dx_t/dt = v_\theta(x_t, t, c)$

$x_1 \rightarrow x_0$ in N steps = N NFEs

A deterministic sampler (DDIM, Euler) makes noise→image a **fixed map**. Later slides use the flow-matching convention $\alpha_t = 1 - t$, $\sigma_t = t$, $v = \varepsilon - x_0$.

Classifier-free guidance

$$\tilde{\varepsilon} = \varepsilon_\theta(x_t, \varnothing) + w \cdot (\varepsilon_\theta(x_t, c) - \varepsilon_\theta(x_t, \varnothing))$$

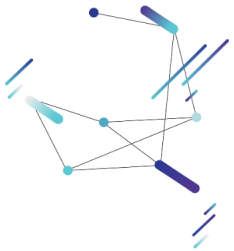
One network, trained with the condition dropped ~10% of the time, gives both terms. w > 1 sharpens and follows the prompt better, at some cost in diversity.

2 × N

NFEs per sample with CFG

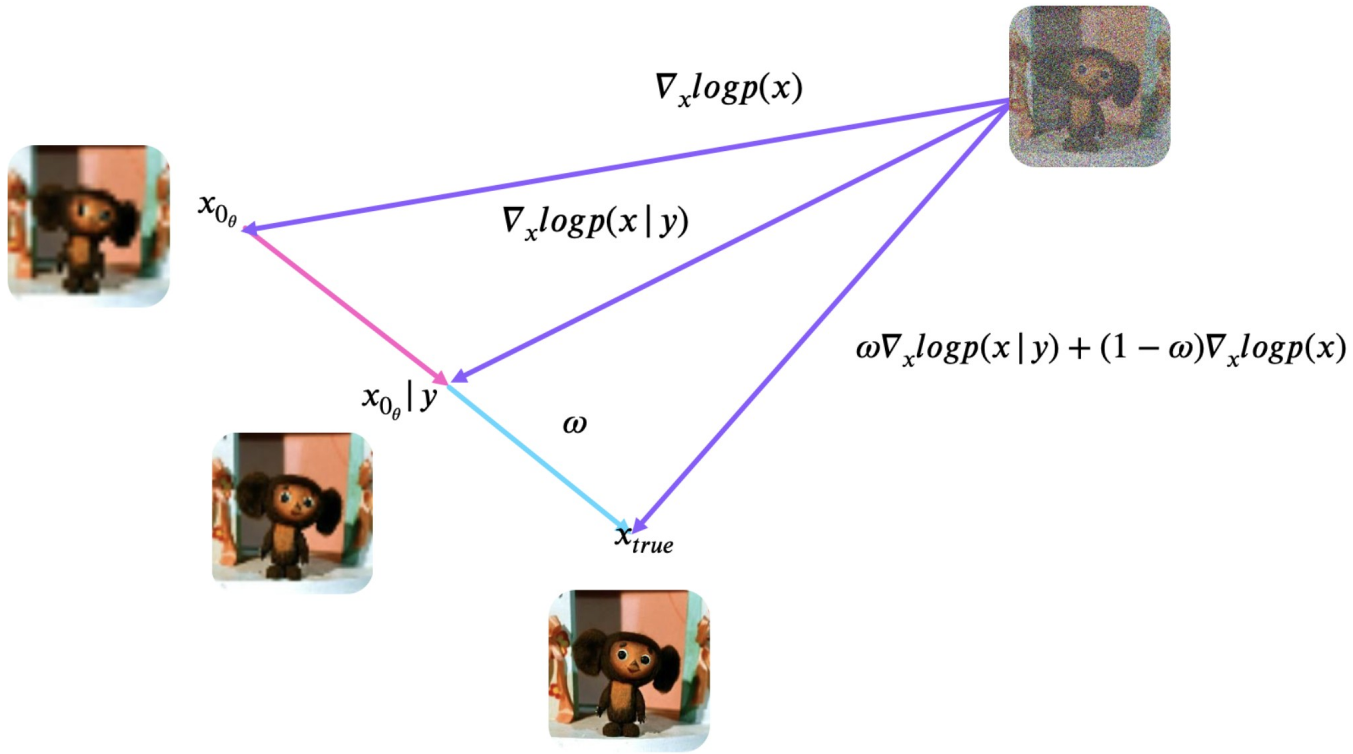
Why it matters today

We distill the **guided** ODE; students bake w in (guidance distillation, LCM, DMD's guided real score).



THE PROBLEM

Slow inference: classifier-free guidance doubles it



$$\tilde{\varepsilon} = \varepsilon(x_t, \varnothing) + w \cdot (\varepsilon(x_t, y) - \varepsilon(x_t, \varnothing))$$

Unconditional and conditional predictions point to different images; CFG **extrapolates** past the conditional one by w.

2 network calls per step → a 50-step sampler costs ~100 forward passes.

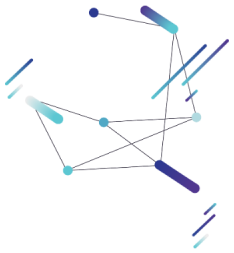
Distilled students bake w in: 1 call per step.

MAP OF THE LECTURE

Five families of distillation

Family	Matches	Training signal	Typical NFE
00 Early trajectory	Teacher ODE solutions	Regression on offline pairs or 2 teacher steps	1–8
01 Consistency & flow maps	Teacher's noise→data map	Self-consistency, flow-map identities	1–4
02 Adversarial	Real or teacher distribution	Discriminator (+ teacher loss)	1–4
03 Distribution matching	Teacher distribution (reverse KL)	Real score minus fake score	1–4
04 Parallel decoding	Teacher steps at student states	On-policy regression, no GAN or fake score	4–8

00, 01 and 04 keep the noise→sample pairing and preserve diversity; 02 and 03 trade it for sharpness. Strong recent systems are often **hybrids**.



DLS

00

Early trajectory distillation

Before consistency models: regress a fast student onto the teacher’s ODE, offline or round by round

Knowledge distillation · Progressive Distillation · Rectified flow & reflow · InstaFlow · Rectified Diffusion

Knowledge distillation: noise → image in one step

$x_T \sim N(0, I)$
fixed noise

→

Teacher: DDIM, N deterministic steps
run once, offline, for every training pair

→

x_0 = teacher sample
target

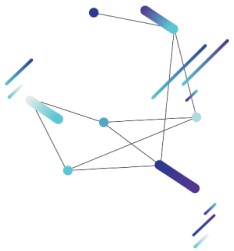
Student G_θ learns the same map $x_T \to x_0$ in a single forward pass

$$L(\theta) = E_{x_T} \| G_\theta(x_T) - \mathrm{DDIM}_\varphi(x_T) \|^2 \quad \text{pairs precomputed}$$

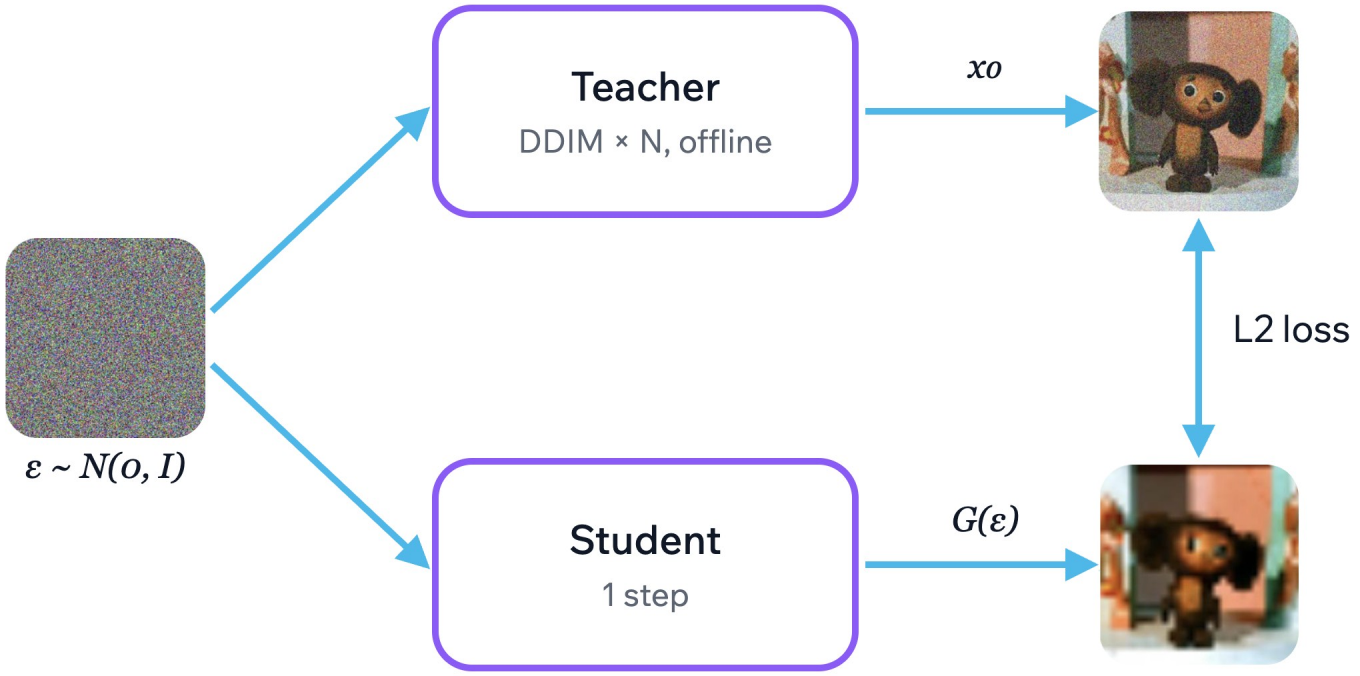
Plus. Dead simple: supervised regression on (noise, image) pairs; any architecture.

Minus. A full ODE solve per pair; the map is highly nonlinear, so one step averages and blurs.

Legacy. The same (x_T, x_0) pairs come back in reflow and Rectified Diffusion.



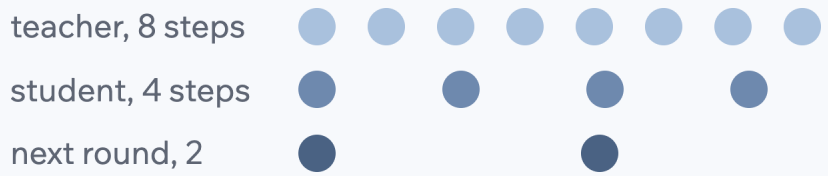
Knowledge distillation



Fixed pair (ε, x_0) from one long teacher solve

Every training pair costs a full teacher solve; one jump over the whole curved trajectory regresses to the mean, so the student is blurry.

Progressive distillation: halve the steps, repeat



two DDIM teacher steps: $z_t \to z_t' \to z_t''$

$$\tilde{x} = (z_t'' - (\sigma_t''/\sigma_t)\cdot z_t) \;/\; (\alpha_t'' - (\sigma_t''/\sigma_t)\cdot \alpha_t)$$

$$L = w(\lambda_t) \cdot \| \tilde{x} - \hat{x}_\theta(z_t) \|^2$$

$\tilde{x}$ = what one student DDIM step must predict to land on z_t''

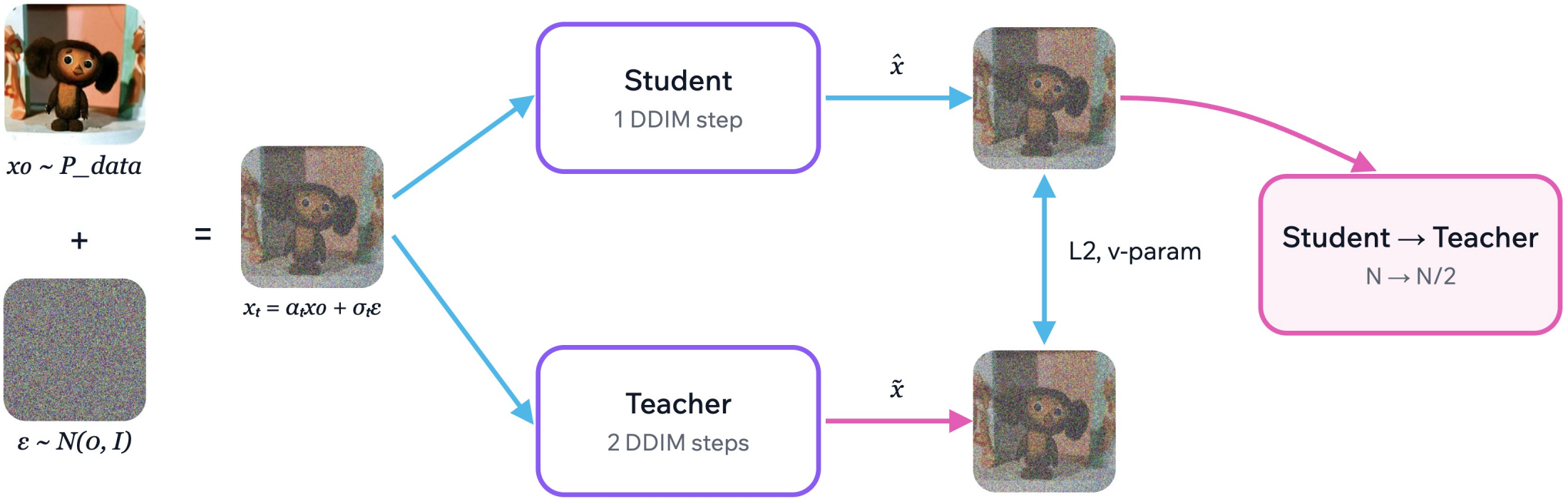
Is it ODE-based? Yes.

The teacher is DDIM with $\eta = 0$, a discretization of the PF-ODE, and the target is the point two deterministic steps away. It differs from consistency models in how the jump is learned: rounds of halving instead of self-consistency.

Side effects

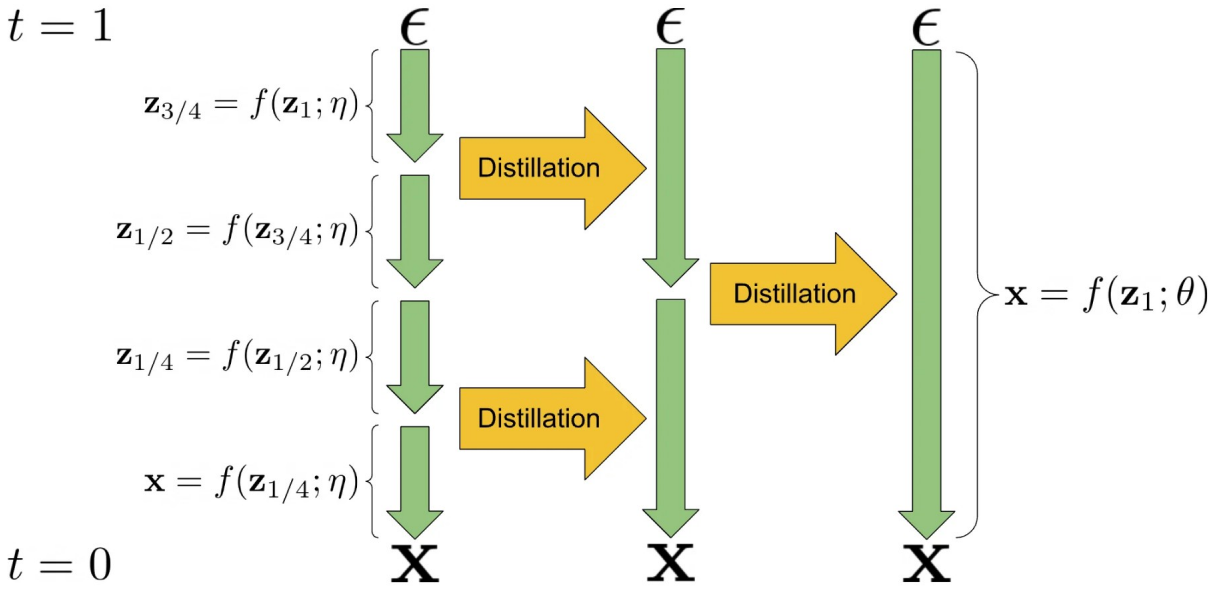
Introduced v-prediction (ε-prediction breaks at low SNR). Guided version: distill CFG into a w-conditioned student, then halve (Meng et al., CVPR 2023).

Progressive distillation



One student step learns two teacher steps; then the student becomes the teacher and the step count halves again: 1024 → … → 4.

Progressive distillation: the details

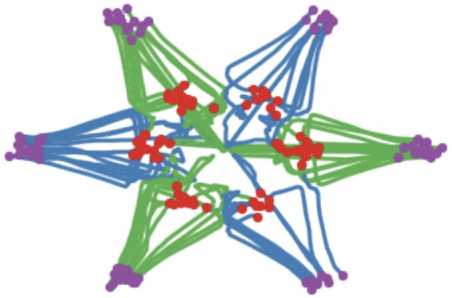


Two rounds: 4 teacher DDIM steps → 2 student steps → 1 step. Each student starts from its teacher's weights.

Details that made it work

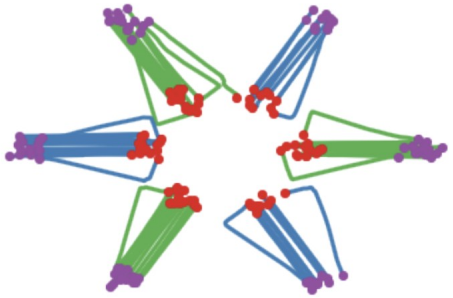
- **Teacher = DDIM with η = 0**: the deterministic PF-ODE sampler; terminal SNR set to 0 ($\alpha_1 = 0$).
- **v-prediction** $v = \alpha_t\varepsilon - \sigma_t x$, $\hat{x} = \alpha_t z_t - \sigma_t \hat{v}$: ε-prediction explodes when SNR → 0 at few steps.
- **Loss weight** "truncated SNR" max(SNR, 1) or SNR + 1, so high-noise steps still get gradient.
- **Schedule**: from up to 8192 steps down to 4; total cost ≤ training the teacher. 4-step CIFAR-10 FID 3.0.
- **Guided models** (Meng et al. 2023): first distill the 2-pass CFG output into one w-conditioned net, then halve.

Rectified flow and reflow: straighten, then distill



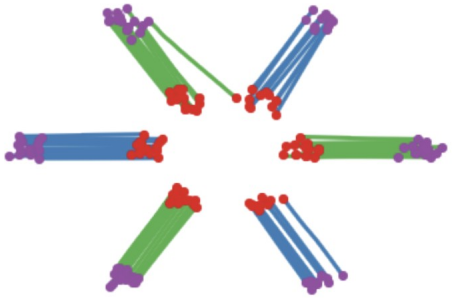
(a)The 1st rectified flow $\boldsymbol{Z}^1$

$\boldsymbol{Z}^1 = \texttt{RectFlow}((X_0, X_1))$



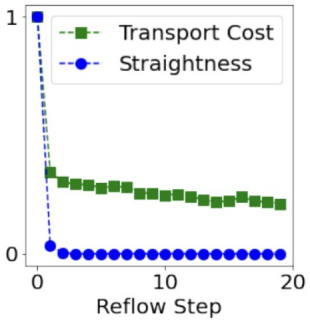
(b) Reflow $\boldsymbol{Z}^2$

$\boldsymbol{Z}^2 = \texttt{RectFlow}((Z_0^1, Z_1^1))$



(c) Reflow $\boldsymbol{Z}^3$

$\boldsymbol{Z}^3 = \texttt{RectFlow}((Z_0^2, Z_1^2))$



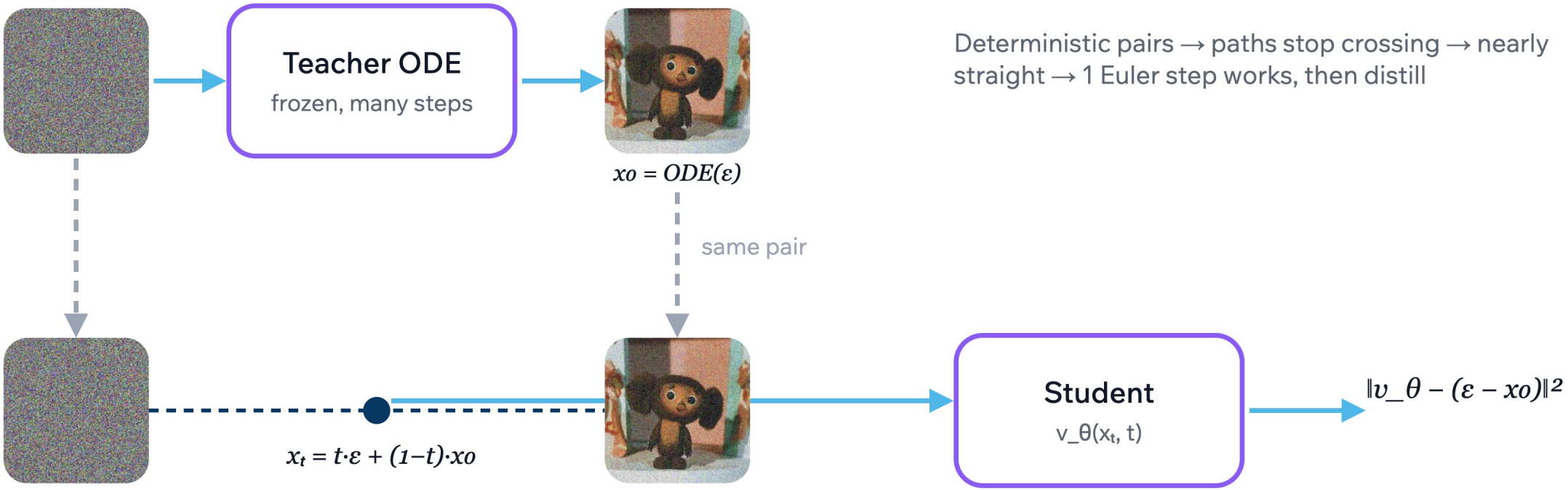
(d) Transport cost, Straightness

```
x_t = t·x_1 + (1 - t)·x_0
min E ‖(x_1 - x_0) - v_θ(x_t, t)‖²
1-RF: independent (x_0, x_1)
reflow: x_1 ~ N(0, I), x_0 = ODE_ψ(x_1)
```

InstaFlow (Liu et al., ICLR 2024)

2-rectified flow on Stable Diffusion 1.5, then knowledge distillation on the straightened pairs: one-step text-to-image in about 0.1 s.

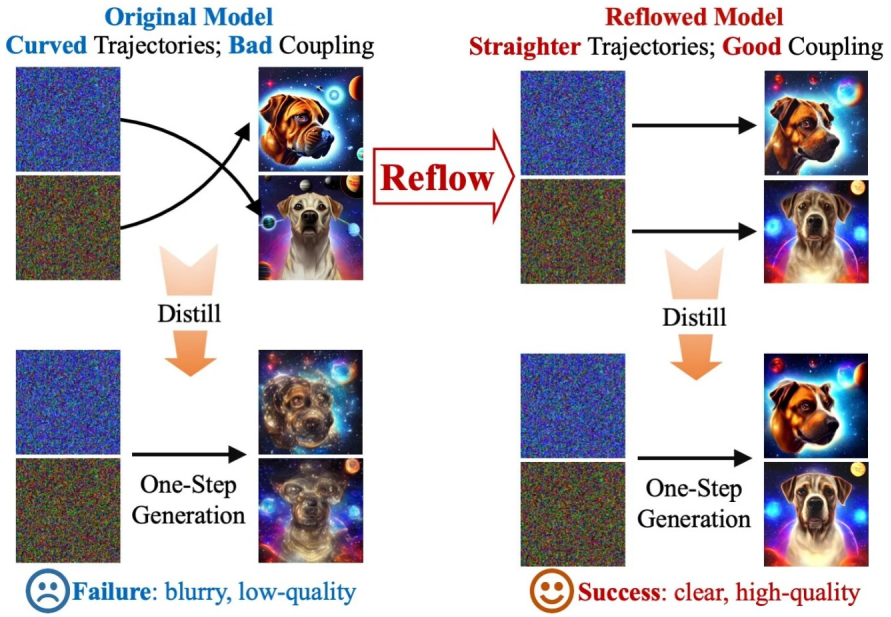
Rectified flow and reflow

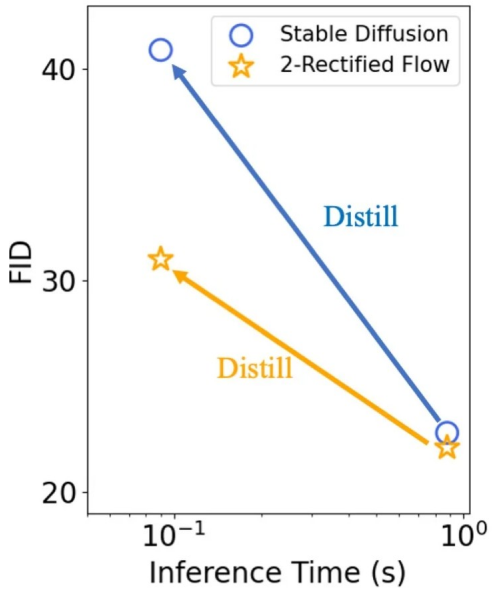


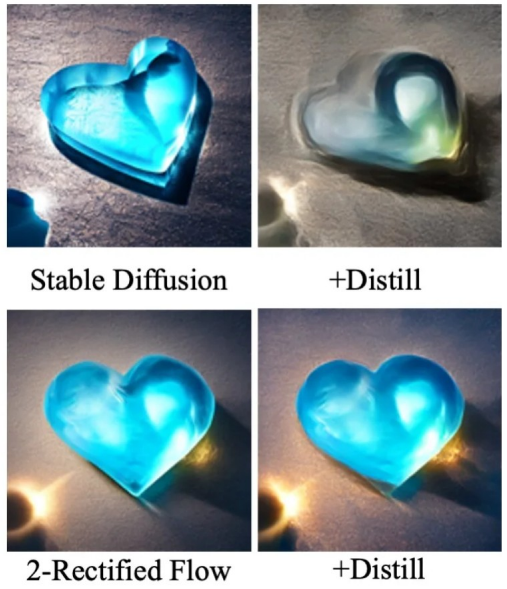
Deterministic pairs → paths stop crossing → nearly straight → 1 Euler step works, then distill

Reflow = retrain the flow on its own (noise, sample) pairs along straight lines; InstaFlow then distills the straightened model into one step.

InstaFlow: why reflow before distilling







37.2 → 23.3 one-step FID-5k on COCO 2017: progressive distillation vs InstaFlow

0.09 s per 512^2 image, InstaFlow-0.9B on an A100; 1.7B variant FID 22.4

199 A100-days total: reflow pairs from SD + 2-RF training + distillation

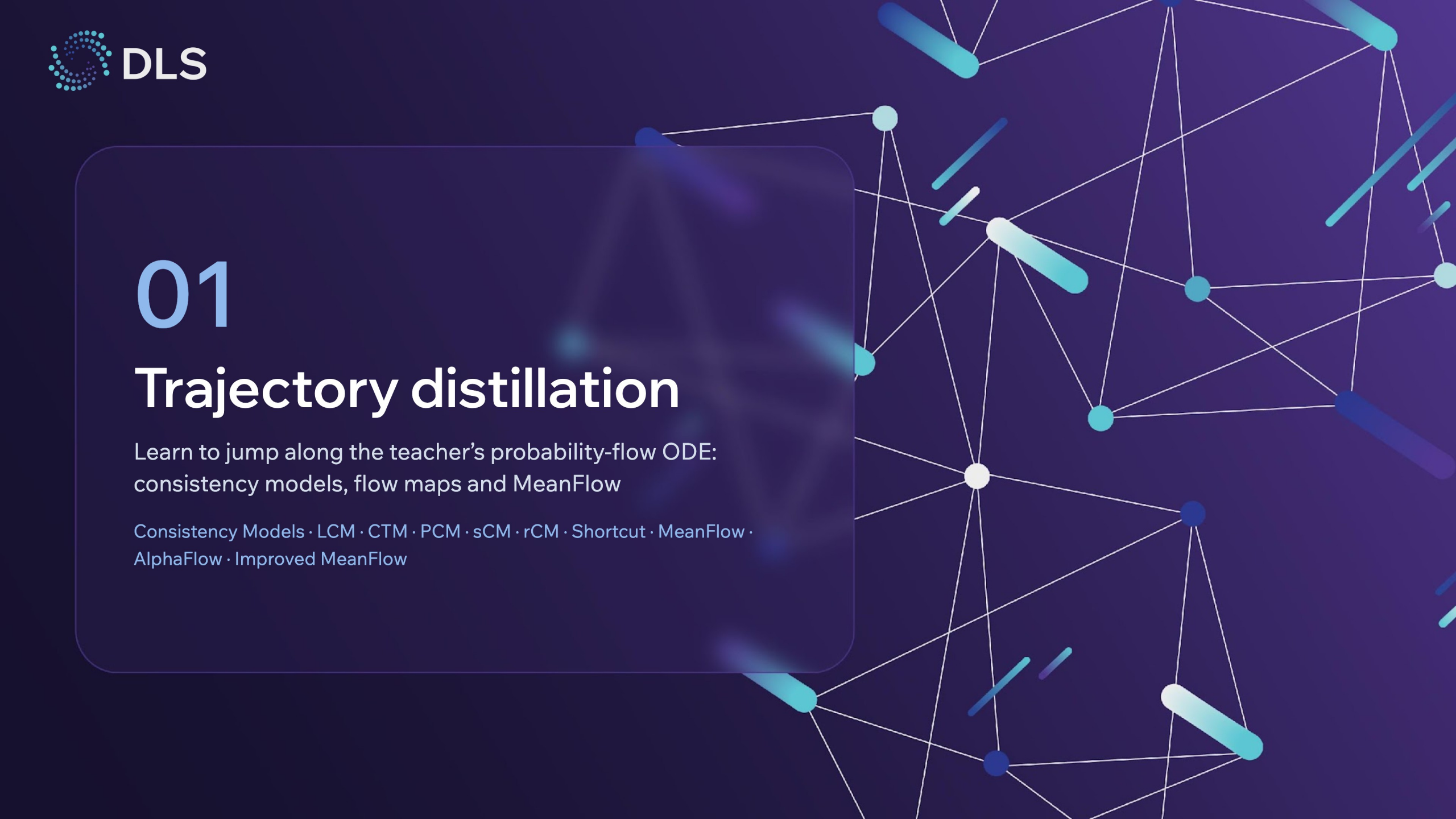
DLS

01

Trajectory distillation

Learn to jump along the teacher's probability-flow ODE: consistency models, flow maps and MeanFlow

Consistency Models · LCM · CTM · PCM · sCM · rCM · Shortcut · MeanFlow · AlphaFlow · Improved MeanFlow

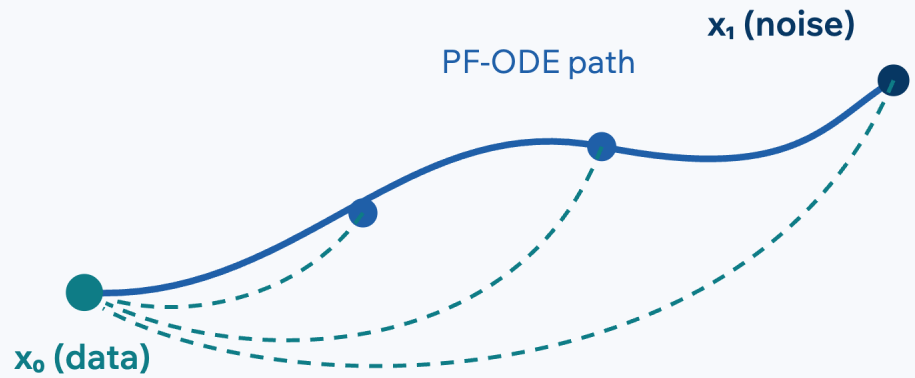


Consistency models: one map for the whole trajectory

Learn $f_\theta(x_t, t) = x_0$ for **every** point on the same PF-ODE trajectory: self-consistency.

```
L_CD = E d( f_θ(x_{n+1}, t_{n+1}),
f_θ⁻(x̂n, t_n) )
x̂n = one teacher ODE step from x_{n+1}
θ⁻ = EMA / stop-grad copy of θ
```

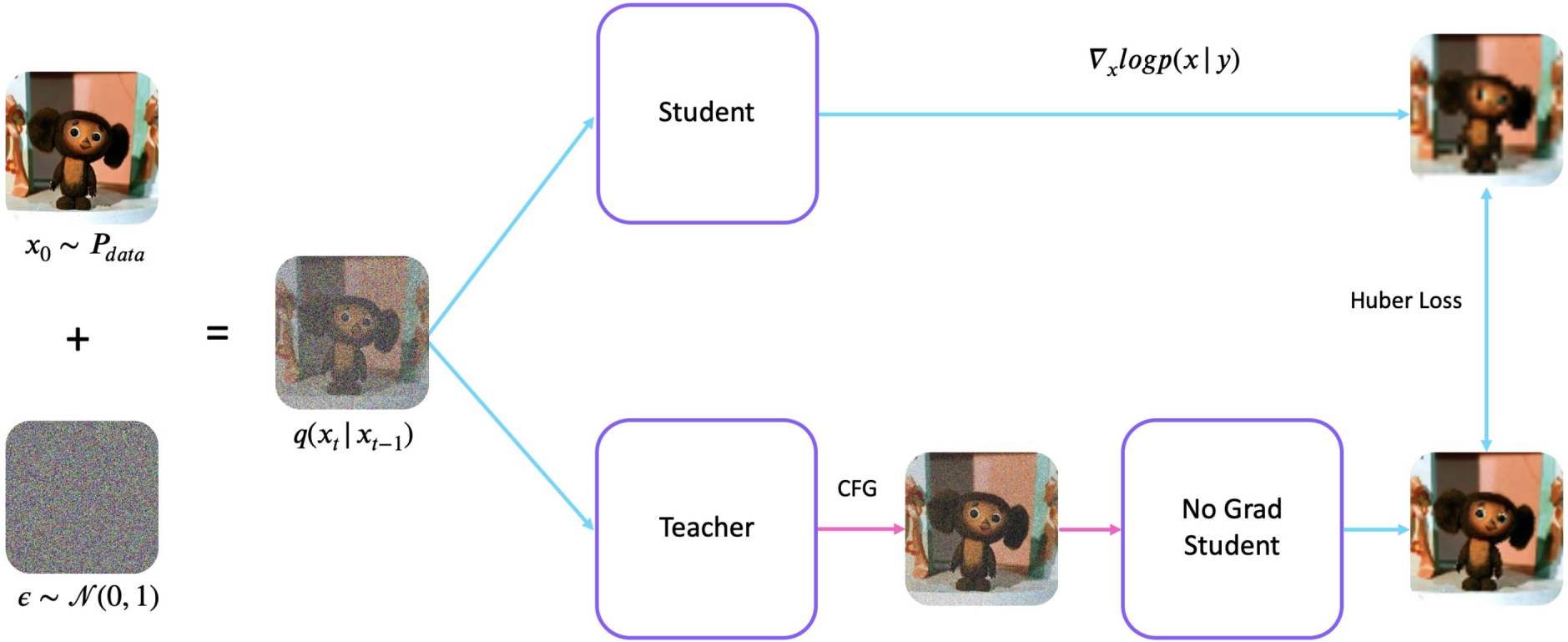
Boundary: $f_\theta = c_{skip}(t)\cdot x + c_{out}(t)\cdot F_\theta(x, t)$, so $f(x, 0) = x$.
Consistency training (CT) drops the teacher: $\hat{x}_n = x_0 + t_n\cdot\varepsilon$ with the same ε.



3.55
1-step FID, CIFAR-10 (CD)

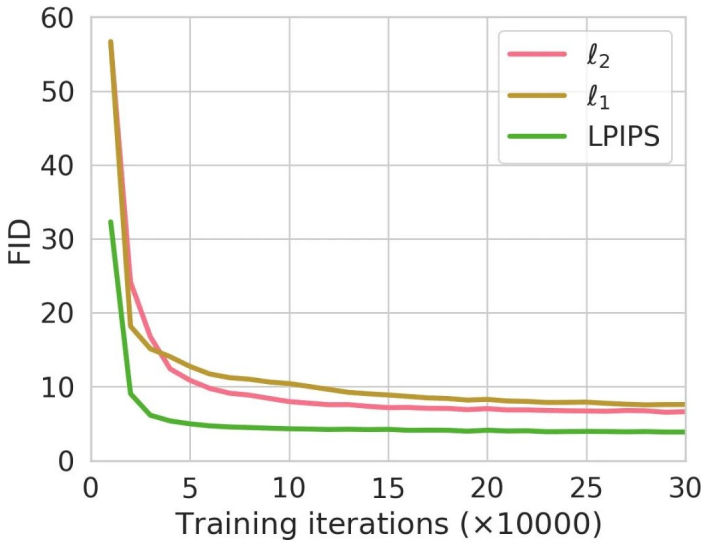
6.20
1-step FID, ImageNet 64 (CD)

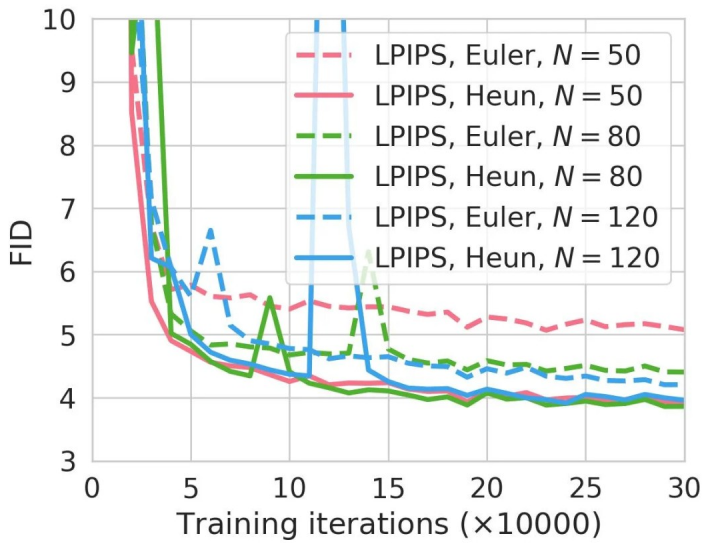
Consistency distillation



- Two points on the same teacher trajectory must map to the same x_0: the target comes from a no-grad (EMA) copy of the student.

Consistency models: the training recipe



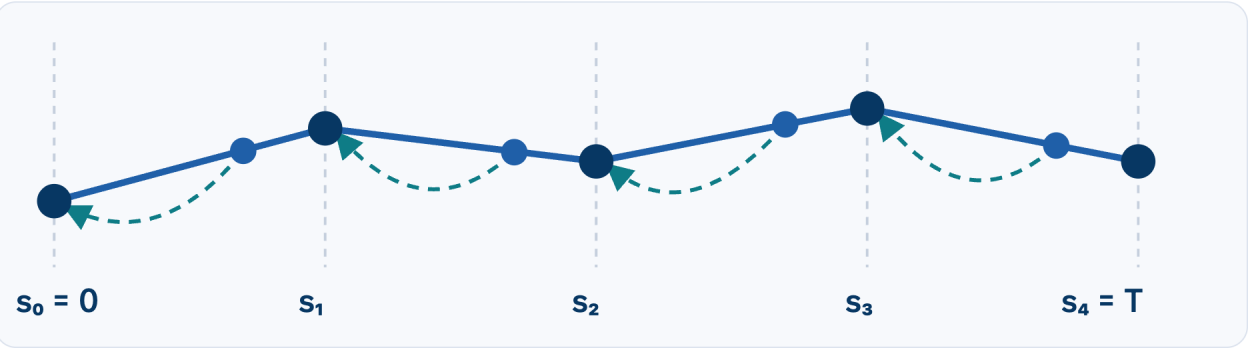


Recipe

- **Boundary by construction**: $c_{skip} = \sigma_d^2/((t-\varepsilon)^2+\sigma_d^2)$, $c_{out} = \sigma_d(t-\varepsilon)/\sqrt{(\sigma_d^2+t^2)}$, $\sigma_d = 0.5$.
- **Metric d = LPIPS** beats ℓ_2 and ℓ_1 (left).
- **Heun** teacher step; N = 18 on CIFAR-10 (right).
- **θ^- = EMA**, $\mu = 0.95$; CT needs N(k), μ(k) curricula.
- **Sampling**: denoise, re-noise by $\sqrt{(\tau^2-\varepsilon^2)}$, denoise.

Bonus: zero-shot inpainting, colorization and super-resolution by iterative replacement, with no extra training.

Phased consistency models: consistency within phases



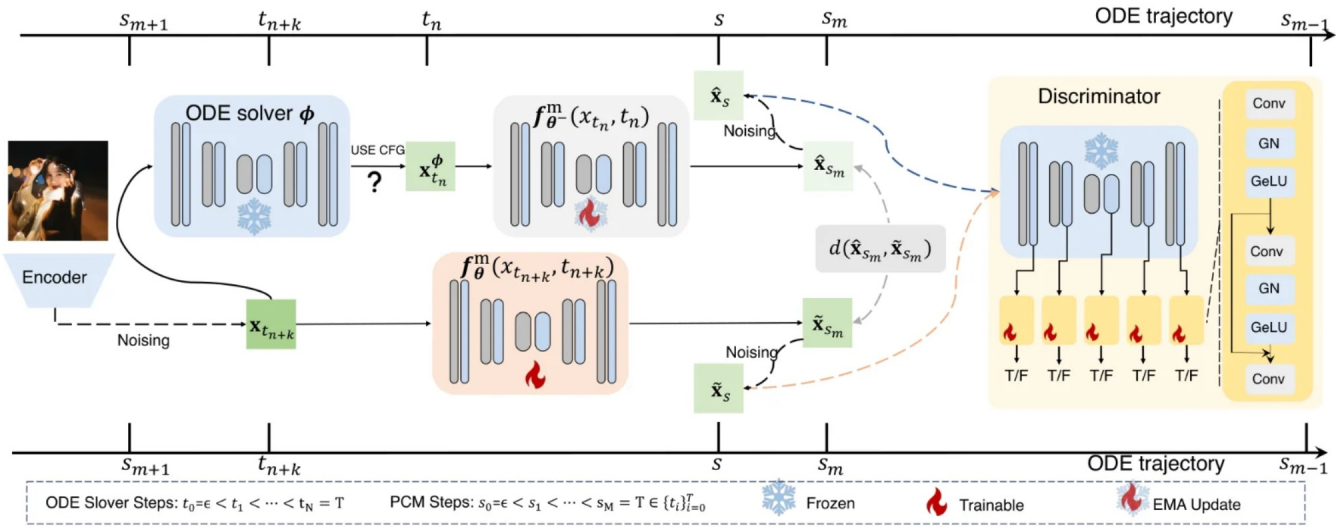

```
t ∈ (s_m, s_{m+1}]: f_θ(x_t, t) → x_{s_m}, not x_0
L = d(f_θ(x_{n+1}, t_{n+1}), f_θ⁻(x̂n, t_n)), same phase
sample: s_M → s_{M-1} → … → s_0, M deterministic jumps
```

What it fixes in LCM

- **Determinism:** CM multi-step sampling re-noises between steps; PCM just jumps edge to edge along the ODE.
- **Guidance:** a wider usable CFG range and negative prompts.
- **1–2 steps:** an optional latent adversarial loss sharpens the fewest-step settings.

Result: SD 1.5 and SDXL from 1 to 16 steps, plus video. M = 1 is a plain consistency model.

PCM: training recipe and numbers



Hacks

- **M = 4 phases** during training.
- **DDIM-form** parameterization: boundary holds exactly, no c_skip / c_out.
- **Adversarial consistency loss**, λ = 0.1: teacher-feature latent discriminator, hinge loss.
- **CFG-free** distillation works for ≥ 4 steps (no w·w′ over-guidance).

FID-SD, COCO-30K	1 step	2	4	8	16
PCM	8.27	9.79	5.81	5.00	4.70
LCM	53.43	11.03	6.66	6.62	7.56

Consistency-style variants

LCM

Luo et al., 2023

Consistency distillation in Stable Diffusion's latent space, on the CFG-augmented ODE, with skipped steps. LCM-LoRA: a plug-in adapter, 2–4 steps.

iCT / ECT

Song & Dhariwal 2023 · Geng et al. 2024

Teacher-free consistency training made practical: pseudo-Huber loss, lognormal noise, step curriculum. ECT starts from a pretrained diffusion model.

CTM

Kim et al., ICLR 2024

Consistency trajectory model $G(x_t, t, s)$: jump from any t to any s. Adds a GAN loss; 1-step FID 1.92 on ImageNet 64.

PCM

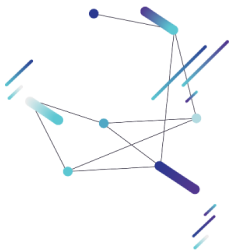
Wang et al., NeurIPS 2024

Phased consistency: split the trajectory into phases and enforce consistency within each, giving deterministic multi-step sampling.

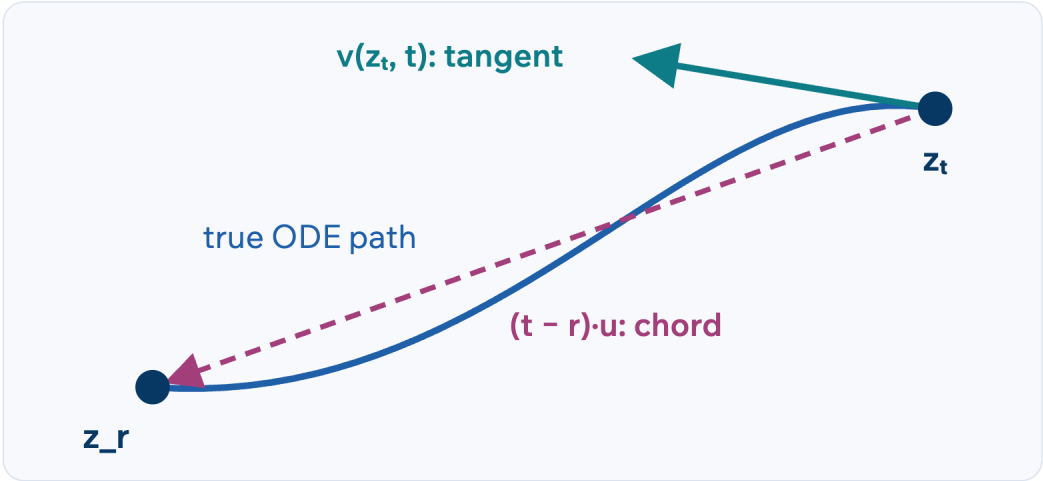
sCM

Lu & Song, ICLR 2025

Continuous-time limit ($\Delta t \to 0$) via TrigFlow and a JVP; stabilized training at 1.5B parameters. 2-step FID 1.88 on ImageNet 512.



MeanFlow: learn the average velocity



Flow matching learns the **tangent** v, so big steps fall off a curved path. MeanFlow learns the **chord**: the average velocity u over [r, t], so one step $z_r = z_t - (t - r)\cdot u$ is exact.

① definition

$$(t - r)\cdot u(z_t, r, t) = \int_r^t v(z_\tau, \tau)\, d\tau$$

② differentiate in t (r fixed)

$$u + (t - r)\cdot d/dt\ u = v(z_t, t)$$

③ MeanFlow identity

$$u = v - (t - r)\cdot(v\cdot \partial_z u + \partial_t u)$$

The right side of ③, with stop-gradient, is the training target. The total derivative is one JVP along (v, 0, 1); no integral is ever computed.

r = t
u = v: plain flow matching

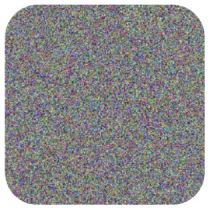
r = 0, t = 1
One-step generator

FID 3.43 / 2.20
1 / 2 NFE, IN-256

MeanFlow

$xo \sim P_data$

+



$\varepsilon \sim N(o, I)$

=

$x_t = \alpha_t xo + \sigma_t \varepsilon$

$v = \varepsilon - xo$

Network

$u_\theta(z_t, r, t)$

u

L2 loss

JVP

tangent (v, 0, 1)

u_tgt = sg(v − (t − r)·du/dt)

One network, one JVP: the target is built from the network itself, so no teacher is needed; plug in $v_teacher(z_t, t)$ to turn it into distillation.

MeanFlow in code: training and sampling

```
# one training step
def train_step(x, c):
  t, r = sample_t_r()           # logit-normal, t ≥ r
  e = randn_like(x)
  z = (1 - t) * x + t * e        # point at time t
  v = e - x                      # conditional velocity
  # distill: v = v_teacher(z, t, c)
  # CFG:     v = w*v + (1-w)*u_θ(z, t, t, ø)
  u, dudt = jvp(lambda z, r, t: u_θ(z, r, t, c),
                (z, r, t), (v, 0, 1))
  u_tgt = v - (t - r) * dudt     # MeanFlow identity
  err = u - stopgrad(u_tgt)
  w = stopgrad(1 / (‖err‖² + 1e-3) ** p)
  return mean(w * ‖err‖²)
```

```
# 1-step or few-step sampling
def sample(steps=1):
  z = randn(shape); t = 1.0
  for r in linspace(1, 0, steps+1)[1:]:
    z = z - (t - r) * u_θ(z, r, t, c)
    t = r
  return z
```

- **(t, r):** two logit-normal draws, swapped so t ≥ r; r = t for ~75% of the batch.
- **jvp:** tangent (v, 0, 1) is d/dt of (z, r, t); target is stop-grad, so no double backprop.
- **w:** adaptive weight, p = 1 works best; a robust, pseudo-Huber-like loss.

MeanFlow variants and relatives

Shortcut models

Frans et al., ICLR 2025

Condition on step size d. Self-consistency: one 2d-step = average of two d-steps. No JVP; one model for 1 to 128 steps.

AlphaFlow

Oct 2025

MeanFlow's flow-matching and consistency terms conflict; an α-Flow curriculum moves from FM to MeanFlow. IN-256 FID 2.58 (1 NFE), 2.15 (2 NFE).

Improved MeanFlow

Geng et al., Dec 2025

Regress on v through the u-parameterization; the guidance scale is an input set at test time. IN-256 FID 1.72 at 1 NFE, from scratch.

SplitMeanFlow

2025

Swap the differential identity for interval additivity: the mean over [r, t] is the length-weighted mean of two sub-intervals.
No JVPs.

Align Your Flow

Sabour, Fidler & Kreis, 2025

Continuous-time flow-map distillation objectives, plus autoguidance and a short adversarial fine-tune; applied to text-to-image.

Common thread

All learn u(x, t, s); they differ in how the big-jump target is built (derivative, splitting, teacher), which sets cost and stability.

DLS

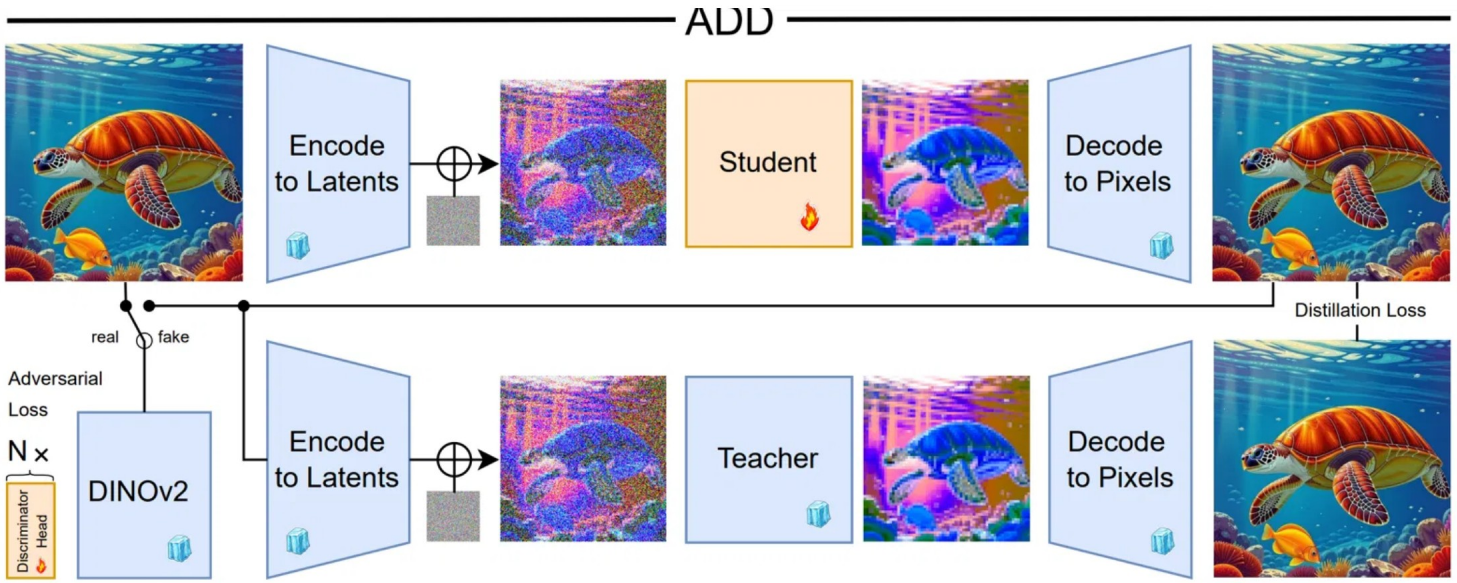
02

Adversarial distillation

Make few-step samples indistinguishable to a discriminator, optionally guided by the teacher

ADD · LADD · UFOGen · SDXL-Lightning · Seaweed-APT · SANA-Sprint · FLUX.1 [schnell]

ADD: adversarial loss + score distillation



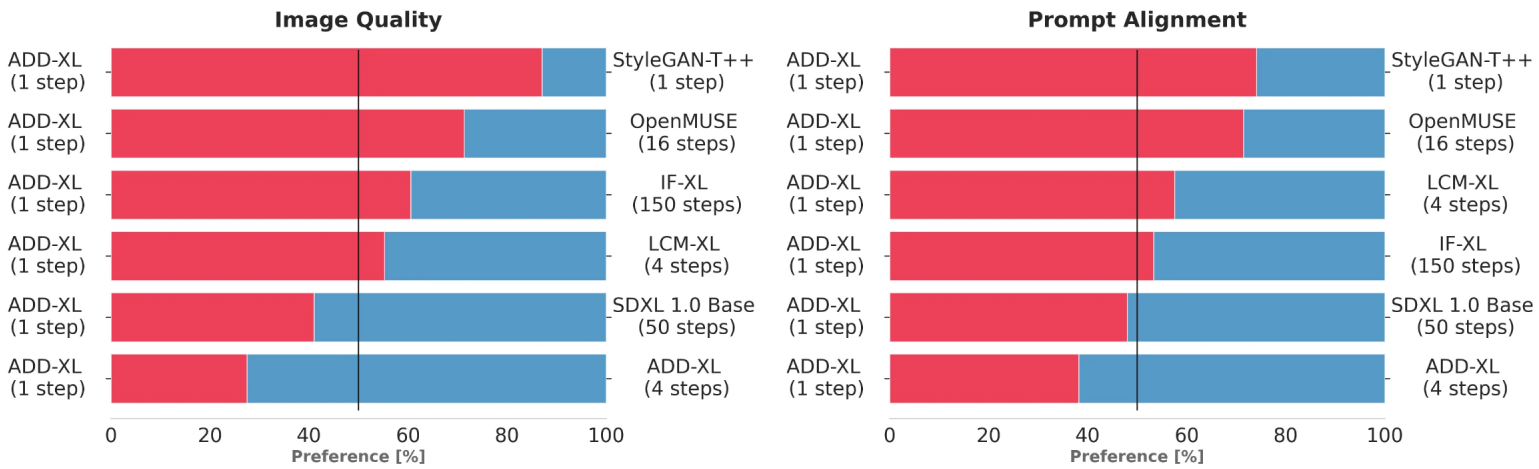
Two roles. The discriminator buys sharpness and realism; **the teacher term prevents overfitting**

Result. SDXL-Turbo and SD-Turbo: 1–4 steps at 512^2, real-time text-to-image.

Limits. The pixel-space discriminator needs VAE decoding: costly at high resolution.

$$L = L_{adv}(\hat{x}_\theta, D) + \lambda \cdot c(t) \cdot \| \hat{x}_\theta - sg(\hat{x}_\psi(\hat{x}_{\{\theta,t\}}, t)) \|^2$$

ADD: details and user study

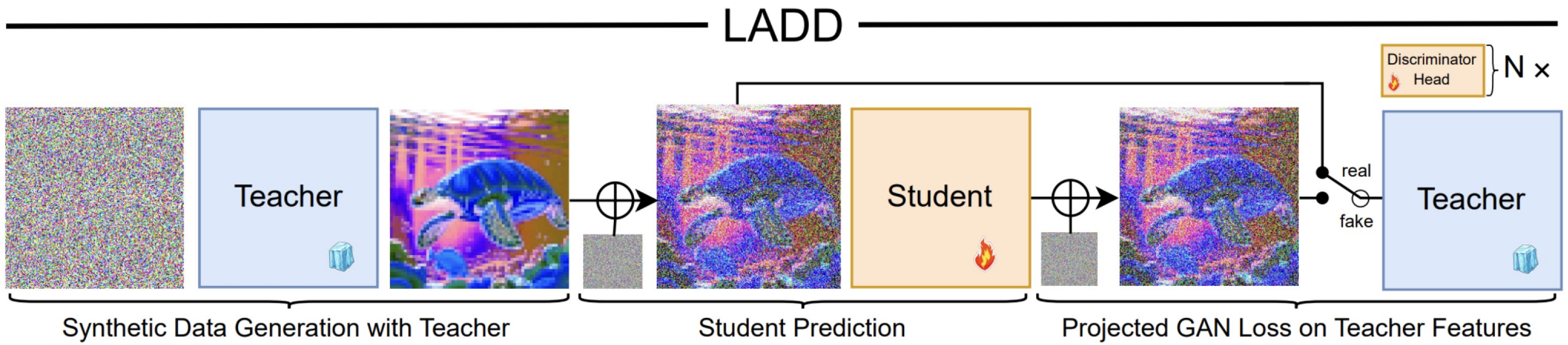


Details

- **Student timesteps** $\tau \in \{999, 749, 499, 249\}$; inputs are noised real images.
- **Discriminator**: frozen DINOv2 ViT-S, trainable heads on several layers, conditioned on text embeddings; hinge + R1.
- **Distillation**: teacher denoises the re-noised student output; weight $c(t) = \alpha_t$, $\lambda = 2.5$.

One step of ADD-XL is preferred over 4-step LCM-XL; with 4 steps it beats the 50-step SDXL teacher.

LADD: the teacher becomes the discriminator



Changes vs ADD

Teacher features on re-noised latents replace DINOv2; teacher samples replace real data; adversarial loss only; no VAE decode.

The noise level is a knob

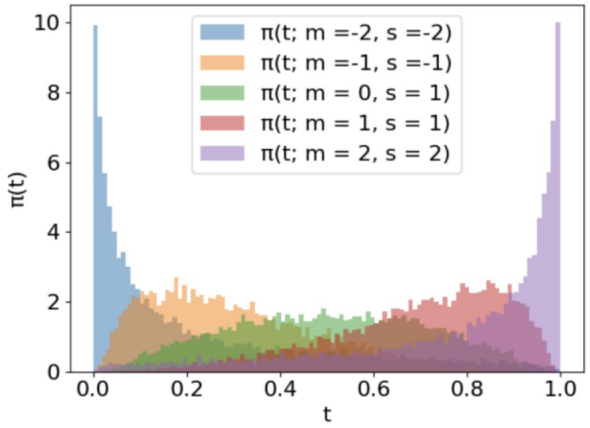
high $\hat{t}$ — low $\hat{t}$

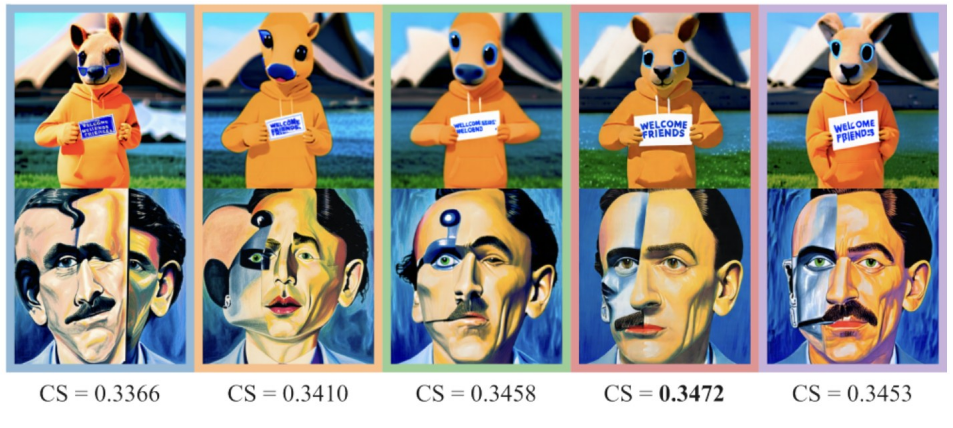
High: global layout and shape. Low: texture and fine detail.

Where it shipped

SD3-Turbo: 4 steps, scales to the 8B SD3, 1024^2 and multi-aspect. FLUX.1 [schnell], 1–4 steps, per its model card.

LADD: re-noising and the training schedule





Re-noise before judging

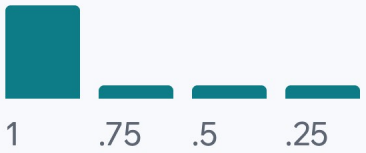
Real (teacher) and fake (student) latents are noised to $\hat{t} \sim \pi(t; m, s)$, a logit-normal, then passed through the frozen teacher.

Low $\hat{t}$: texture. High $\hat{t}$: global structure. Best at m = 1, s = 1.

Stage 1 · warm-up, 500 iters

p = [0, 0, .5, .5]: low noise only, for stability above 512^2

Stage 2 · main training



p = [.7, .1, .1, .1]: mostly full noise, to sharpen 1-step

Inference

Student trained on $t \in \{1, .75, .5, .25\}$. 2 steps: t = 1, 0.5; 4 steps: all four, with a consistency sampler (denoise, re-noise).

LADD vs consistency distillation

0.2712
CLIP score · LCM, 4 steps

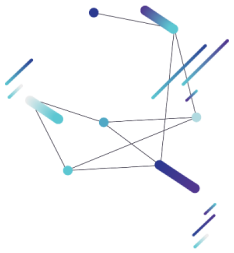
0.3472
CLIP score · LADD, 1 step

With 4× fewer steps, LADD is sharper and follows the prompt, including rendered text. Consistency regression averages and washes out; the adversarial loss on teacher features does not.

Adversarial variants

Method	Year	Key twist
UFOGen	2023	DiffusionGAN-style one-step text-to-image; discriminator on noised samples
SDXL-Lightning	2024	Progressive adversarial distillation down to 1 step; U-Net encoder as latent discriminator
DMD2	2024	GAN head on the fake-score network, trained on real images (see part 03)
Seaweed-APT	2025	Adversarial post-training against real data; one-step 2 s, 720p video
SANA-Sprint	2025	Hybrid: continuous-time consistency (sCM) + LADD, 1–4 steps

Pattern: pure GAN training is rarely used alone. A trajectory or score loss supplies stable, mode-covering guidance; the discriminator adds sharpness.



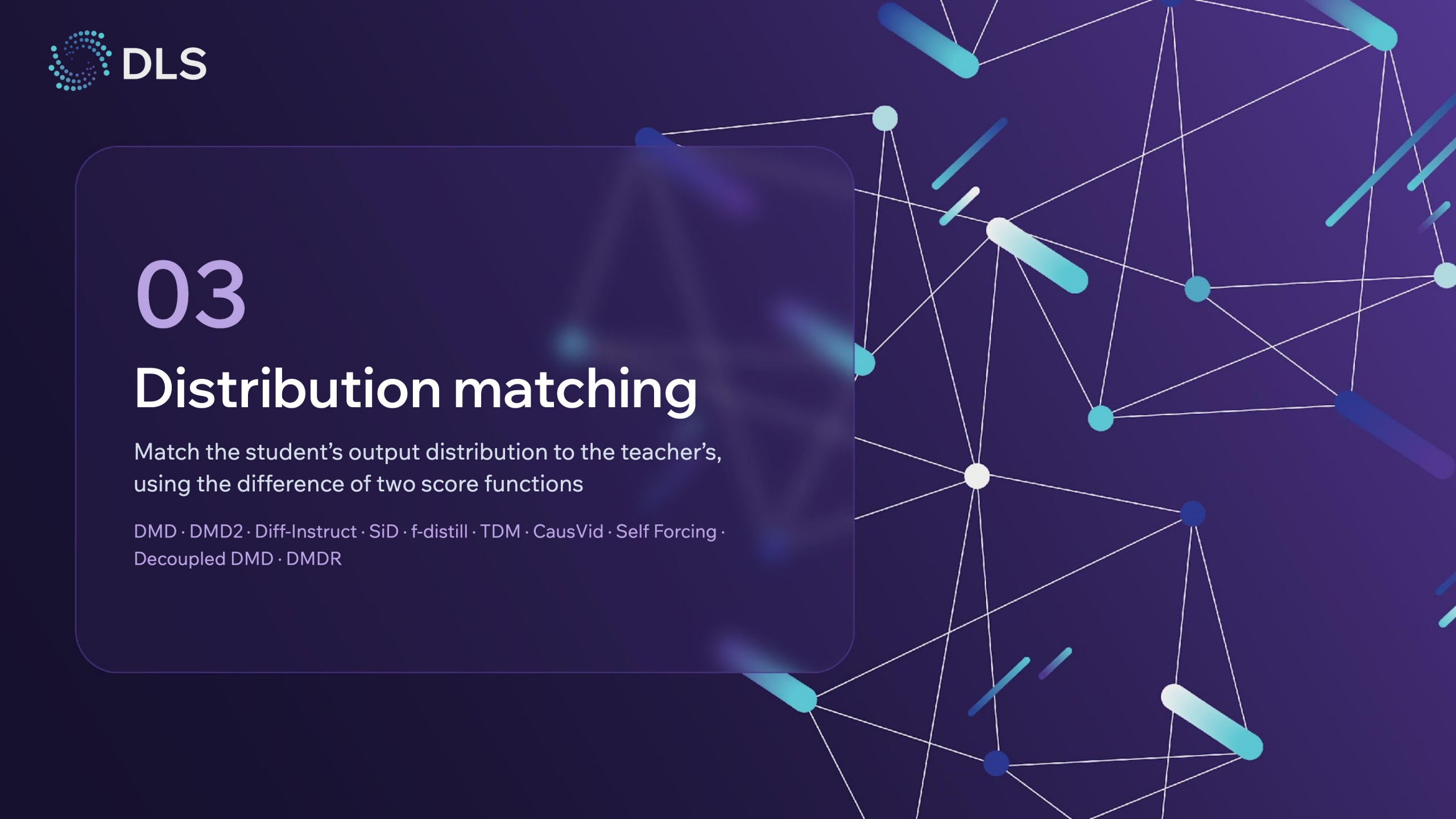
DLS

03

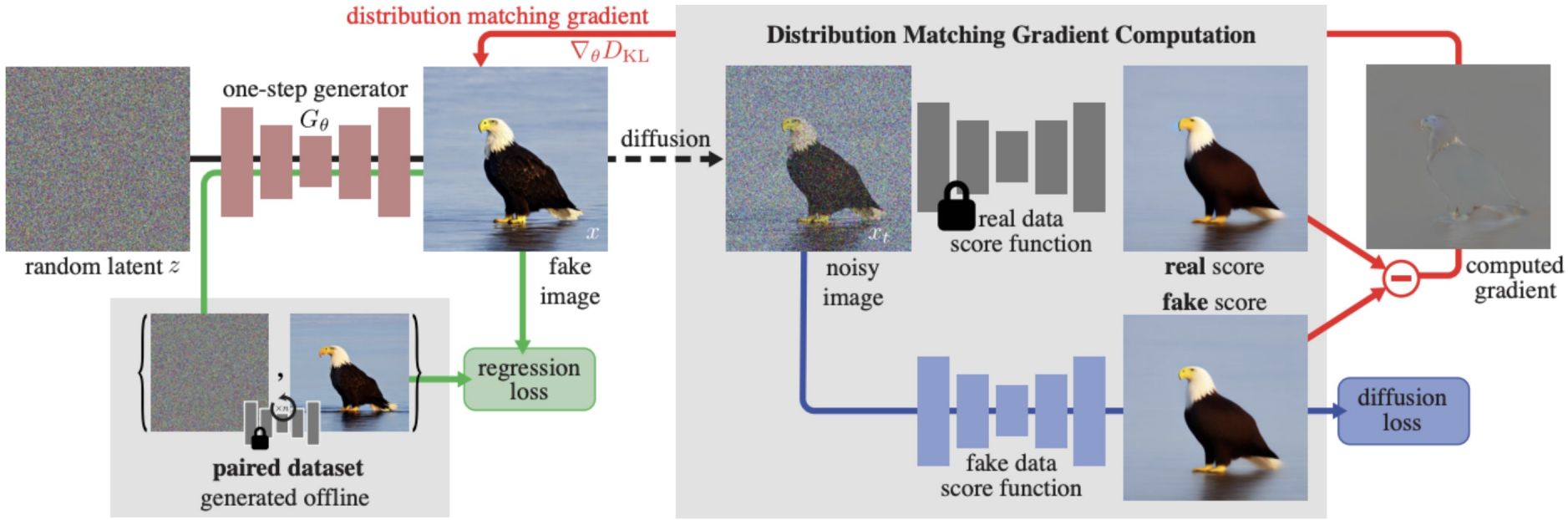
Distribution matching

Match the student's output distribution to the teacher's, using the difference of two score functions

DMD · DMD2 · Diff-Instruct · SiD · f-distill · TDM · CausVid · Self Forcing · Decoupled DMD · DMDR



DMD: a reverse-KL gradient from two scores

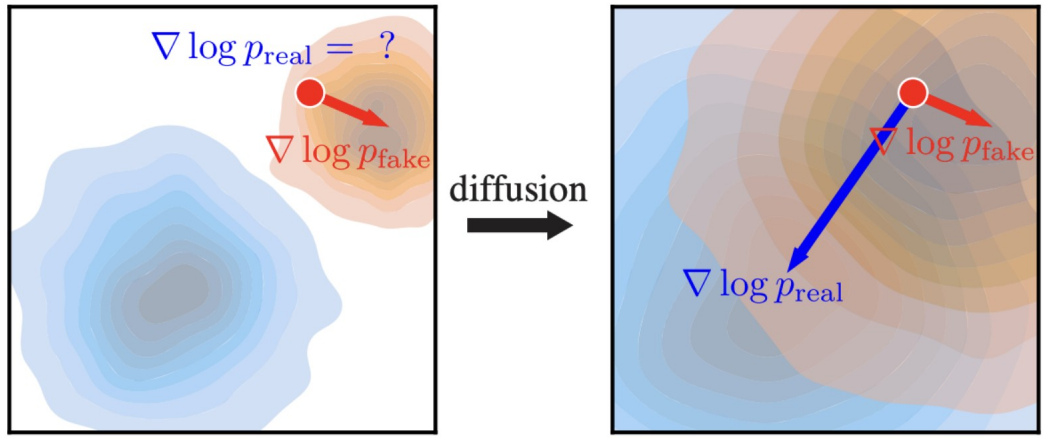


$$\min_\theta \mathbb{E}_t \, D_{KL}(p_{fake,t} \,\|\, p_{real,t})$$

$$\nabla_\theta \approx \mathbb{E}[-w_t \cdot (s_{real}(x_t, t) - s_{fake}(x_t, t)) \cdot \partial G_\theta(z)/\partial \theta]$$

DMD: why diffuse, and why regress

1 · Heavy re-noising makes the scores meet

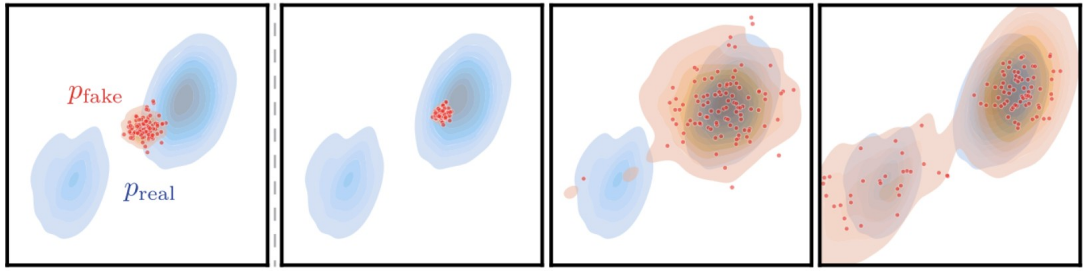


(a) for unperturbed distributions, both scores may not be defined simulateneously everywhere

(b) after diffusion, the distributions overlap, making our objective well-defined

Clean p_real and p_fake barely overlap, so ∇log p_real is undefined at fake samples. After heavy noise they overlap and real − fake becomes a meaningful direction.

2 · Reverse KL is mode-seeking



initial state | (a) real score only | (b) real+fake scores | (c) real+fake scores & regression loss

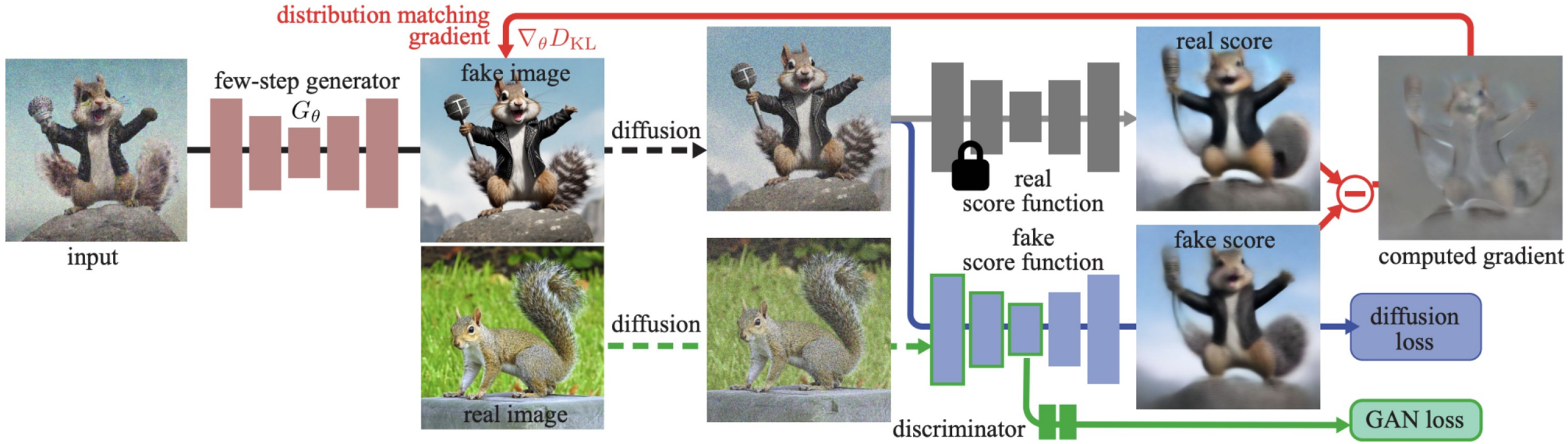
(a) Real score only: samples collapse to a single point.

(b) Real − fake: the fake score repels collapse, but the far mode is still missed.

(c) + regression on offline teacher pairs anchors noise → mode and covers both.

DMD2 later drops (c) for TTUR + a GAN loss on real data.

DMD2: four fixes, no regression loss



1 · No regression
No teacher-ODE dataset needed.

2 · TTUR
~5 fake-score steps per G step.

3 · GAN head
On fake-score bottleneck, real data.

4 · Backward simulation
Train on the student's own states.

DMD2: backward simulation and results

Forward diffusion: domain gap

Training a 4-step student on noised **real** images shows it inputs it never sees at test time, where step k receives the student’s own, imperfect output from step k − 1.

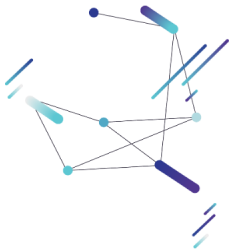
Backward simulation: aligned

During training, run the student’s own chain from pure noise and train each step on those states. Same idea as Self Forcing and PDD’s on-policy targets.

Distribution-matching variants

Method	Divergence or change	Why it matters
Diff-Instruct (2023)	Integral KL over t	Same gradient form as DMD, derived independently
SiD (2024)	Fisher divergence via score identities	Data-free one-step distillation
f-distill (2024)	General f-divergence	Density-ratio weighting trades mode-seeking for coverage
TDM (2025)	Matching along the trajectory	Bridges trajectory and distribution matching for few-step
CausVid, Self Forcing	DMD for autoregressive video	Student rolls out its own context during training
rCM (2025)	sCM + small DMD term	Reverse-KL sharpness on a trajectory backbone

Common risk: reverse-KL objectives are mode-seeking, giving sharp samples with less diversity. Regression, GAN losses on real data and f-divergence weights all push back on this.



Decoupled DMD: what really drives it?

$$\nabla L_{DMD} = -[\ (s_{real}^{c} - s_{fake}^{c}) + (\alpha - 1)(s_{real}^{c} - s_{real}^{u})\ ] \cdot \partial G/\partial \theta$$

c = conditional, u = unconditional, α = CFG scale of the real score

DM TERM · THE SHIELD

Distribution matching

Real minus fake score. Acts as a regularizer: keeps training stable and suppresses over-saturation and artifacts.

CA TERM · THE SPEAR

CFG augmentation

The guidance direction applied straight to the student. The main engine that turns a many-step model into a few-step one.

FIX · DECOUPLE

Separate noise schedules

CA re-noises only to levels cleaner than the current step; DM uses the full range for global fixes. Used for Z-Image's 8-step model.

Trajectory DMD: what exactly is matched

Trajectory distillation: points

$$\| \mathrm{ODE}_\psi(x_t \to x_s) - G_\theta(x_t \to x_s) \|$$

✓ every step supervised

✕ too strict, needs teacher ODE solves

DMD / DMD2: final x_0 only

$$\mathrm{KL}(p_{\theta,0} \| p_{\psi,0})$$

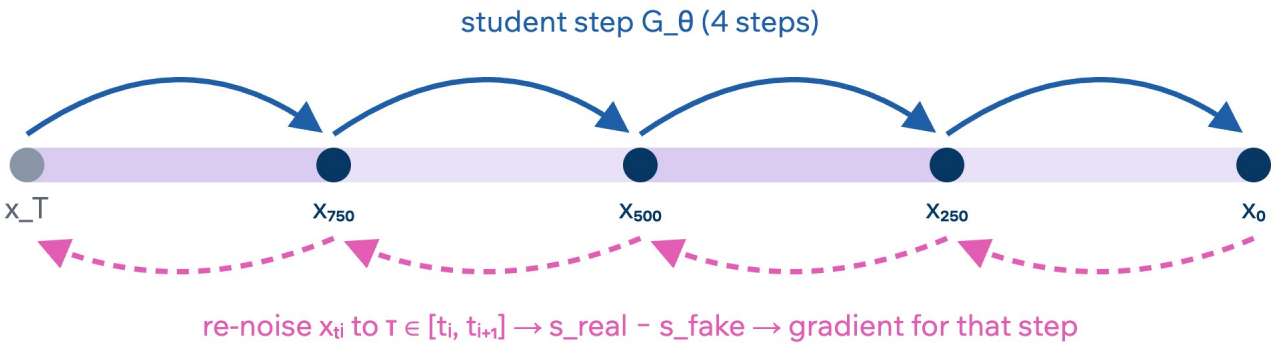
✓ distribution-level, data-free

✕ intermediate states unconstrained

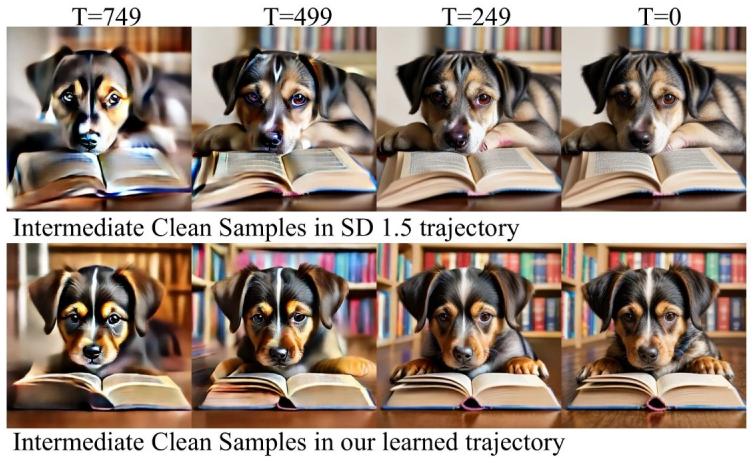
TDM: every step's marginal

$$\Sigma_i \, \mathrm{KL}(p_{\theta,t_i} \| p_{\psi,t_i})$$

✓ both: every step, distribution-level

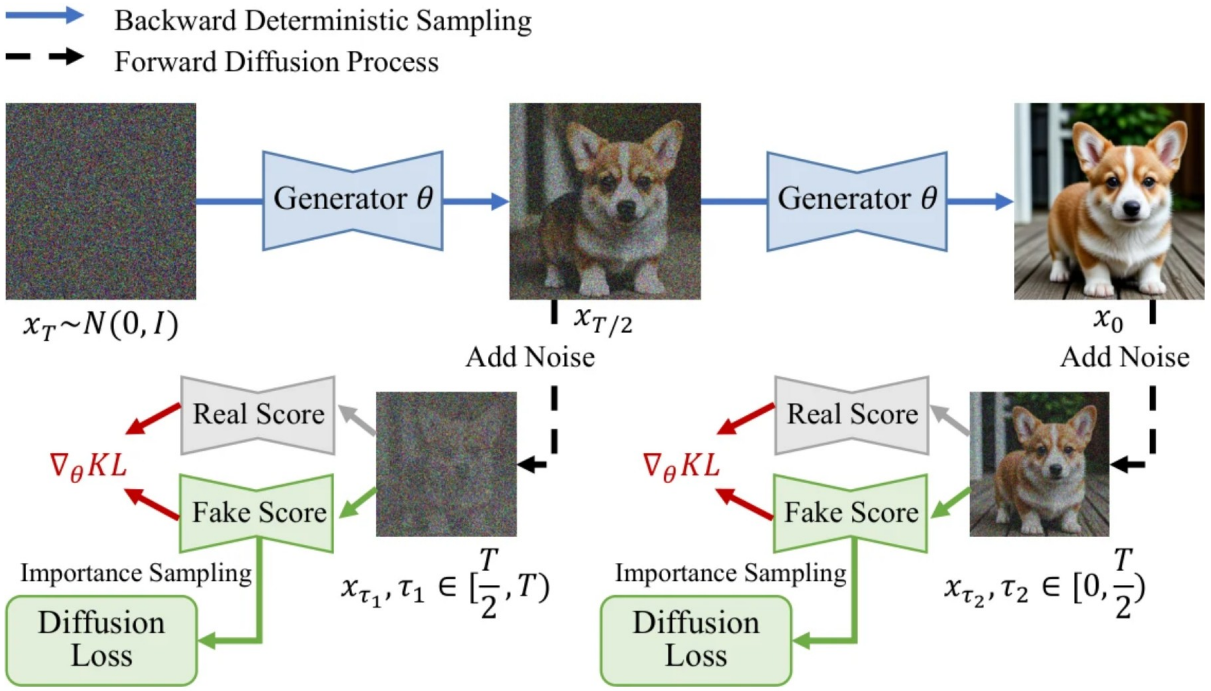


Intervals **do not overlap**, so one fake score serves all steps: τ itself tells which step a sample came from.



Not the teacher's path: only the **marginals** match, so the student's own trajectory can be sharper early.

TDM: trajectory distribution matching



Match the student's **intermediate marginals** to the teacher's diffused marginals, step by step, instead of matching ODE points.

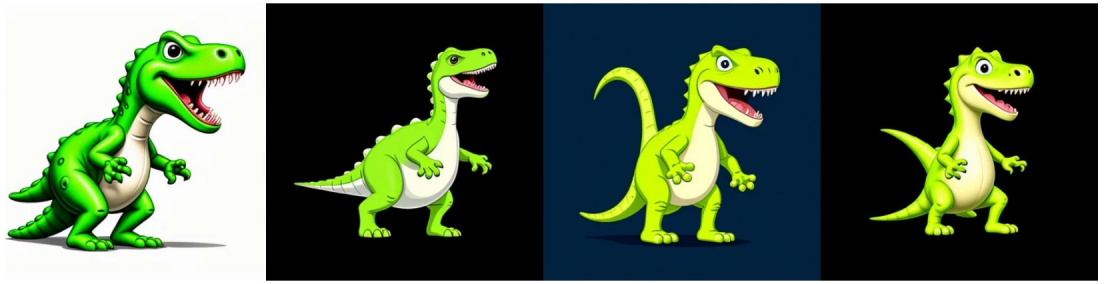
```
for each student step i:
x˜ = sg( x_{tᵢ} + λ(s_real - s_fake) )
L = ‖ x_{tᵢ} - x˜‖ (pseudo-Huber)
noise only within [tᵢ, tᵢ₊₁]
```

- **Data-free**: prompts only, no teacher sampling.
- Targets are **noisy** x_t, not clean $\hat{x}$: deterministic sampling works.
- Backprop through one ODE step only.

TDM: hacks, ablations and cost

Mode coverage, same prompt and seeds:

DMD2

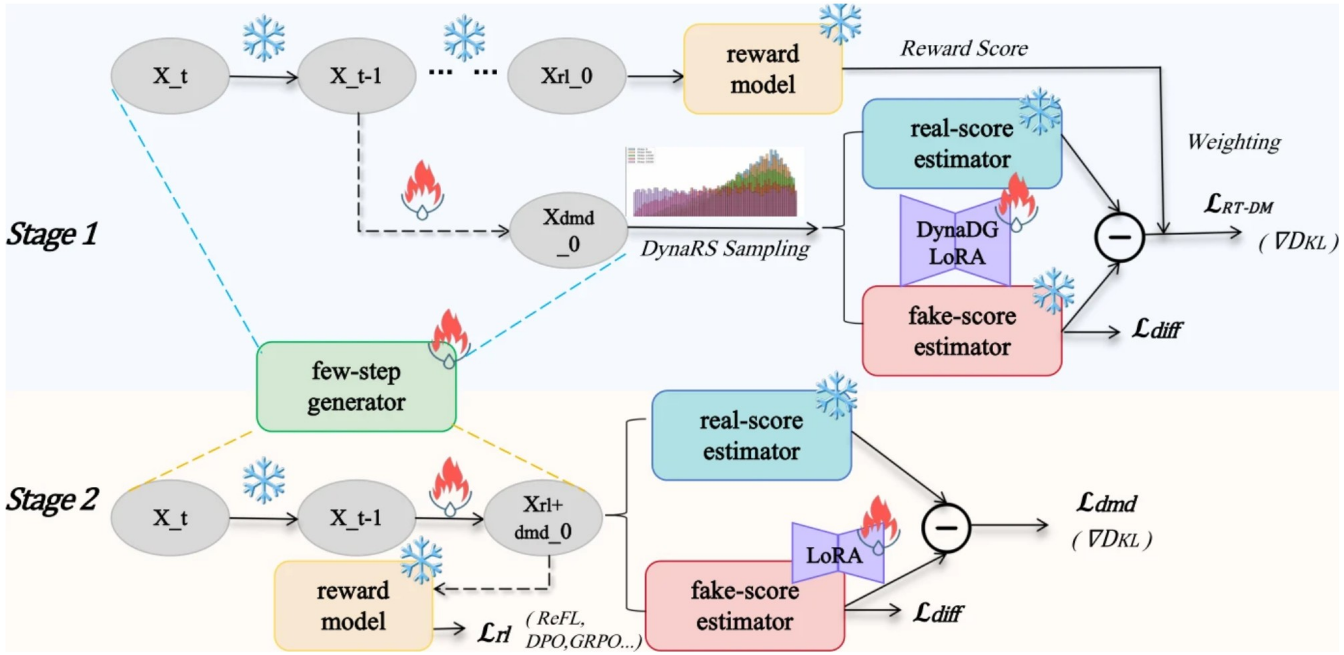


TDM

SD-v1.5, 4 steps	HPS
TDM (full)	31.31
− step-count condition	29.39
− importance sampling	29.27
− pseudo-Huber surrogate	30.85
clean $\hat{x}$ targets	24.63

- **Step-aware**: K is sampled and fed to student and fake score; one model for 1–4 steps.
- **Cost**: SDXL in ~2 A800-days (DMD2: 160 A100-days); PixArt-α in 500 iterations.
- Pseudo-Huber $c = 0.00054 \cdot \sqrt{d}$; 1 fake-score update per step.

DMDR: distribution matching meets RL




```
Stage 1 · reward-tilted DM
∇ = E[ R(x₀)·(s_fake - s_real)·∂x/∂θ ]
R = exp(r(x₀)/β), β = batch max
Stage 2 · joint
L = L_DMD + λ_rl·L_RL, λ_rl = 0.5
```

- RL pulls the student **past the teacher**.
- DMD keeps it near the teacher: a built-in guard against **reward hacking**.
- L_RL is plug-in: ReFL (default), DPO, GRPO.

DMDR: the four key components

Stage 1 · cold start: distill, softly tilted by reward

RT-DM: reward-tilted DM

Problem. Early 4-step samples are incoherent: reward is noisy, direct RL unstable.

Fix. Weight each sample's DMD gradient by its reward.

```
w = exp(r(x₀)/β)
β = max r in batch
```

No backprop through the reward; target tilts to high-reward modes of the teacher.

DynaDG: move p_real

Problem. p_fake and p_real barely overlap, so s_real − s_fake points nowhere useful.

Fix. LoRA on the real-score net pulls the perceived p_real toward p_fake.

```
LoRA scale: 1 → 0
```

Annealed away, so the true teacher is recovered.

DynaRS: noise first

Problem. Same overlap problem, attacked in time instead of space.

Fix. Bias re-noise t toward high noise, where both are ≈ Gaussian.

```
t: high → uniform
```

Global structure first, texture later.

Stage 2 · joint

DMD as the RL regularizer

Problem. RL after distillation anchors to the distilled model, which already lost modes → reward hacking.

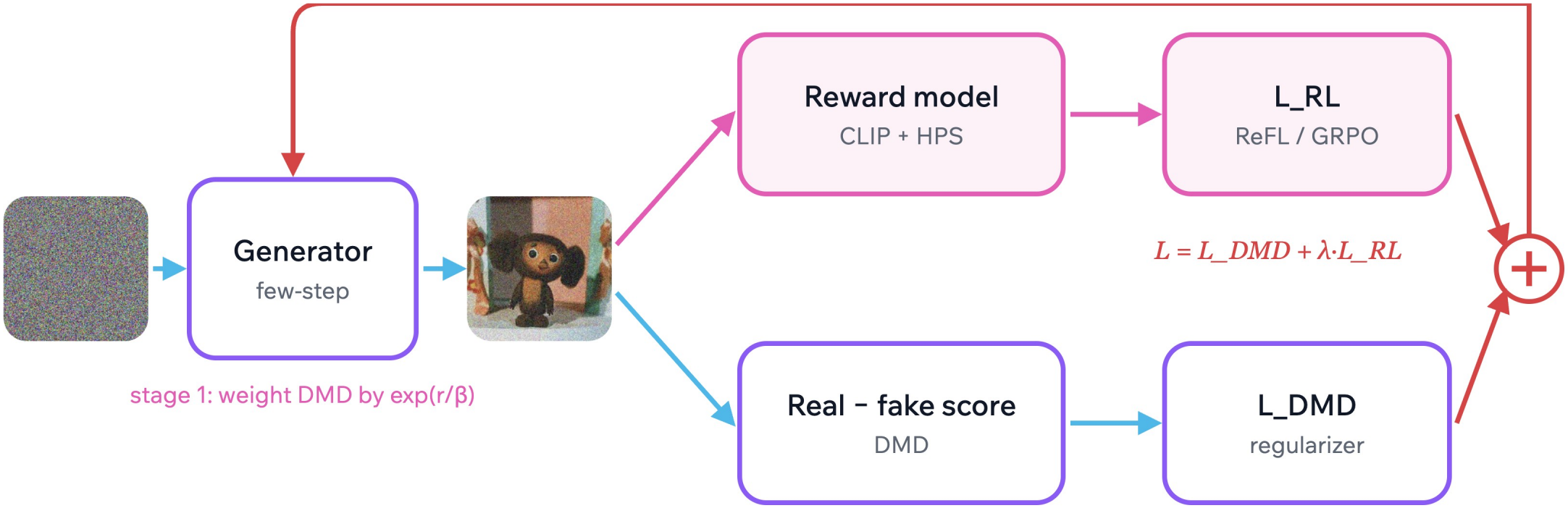
Fix. Train both at once; DMD anchors to the **teacher**.

```
L = L_DMD + λ·L_RL
L_RL: ReFL, GRPO, …
```

RL climbs the reward, DMD projects back onto the teacher's support.

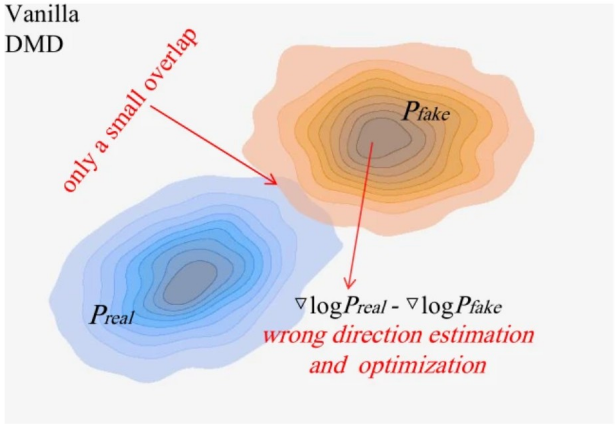
Overlap tricks (DynaDG, DynaRS) are **annealed**: they only help the cold start and vanish by the end of stage 1.

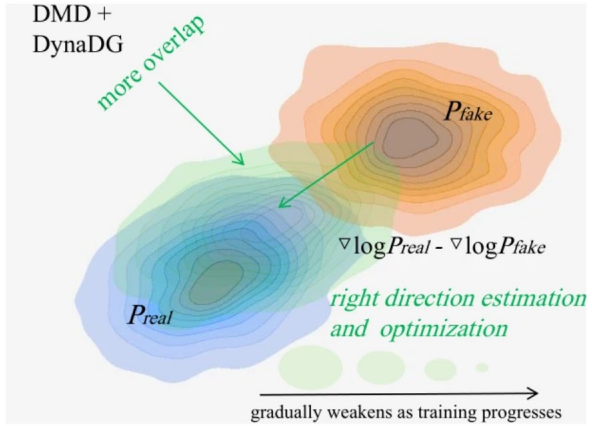
DMDR: DMD + reinforcement learning



RL pushes the student past the teacher toward high reward; the DMD loss keeps it on the teacher's distribution and blocks reward hacking.

DMDR: cold-start hacks and results





(a) Training Reward Score Comparison (b) Test-set Reward Score Comparison (c) DPG Score Comparison

Cold-start hacks

- **DynaDG**: LoRA on the real score shifts p_real toward p_fake; LoRA scale annealed to 0.
- **DynaRS**: re-noise t biased to high noise early, annealed to uniform.

Results, 4 steps vs teacher

- SD3.5-L: GenEval .71 → .72, DPG 84.1 → 85.3
- SDXL HPS 32.98 vs DMD2 28.57
- Cost: diversity, LPIPS .68 → .55

DLS

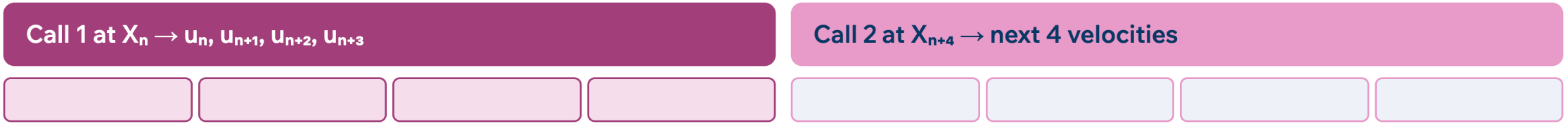
04

Parallel decoding distillation

One call predicts several teacher steps at once, trained on-policy, without VSD or adversarial losses

PDD · Shaul, Liu, Vahdat & Berner (NVIDIA, Weizmann Institute) · arXiv, July 2026

PDD: a block of mean velocities per call



Time grid $t_0 < t_1 < \ldots < t_N$, grouped into blocks of L sub-intervals (here L = 4): one network call per block

$$u_k = W_k \cdot h_\theta(X_n, t_n),\ k = n \ldots n+L-1$$
$$\bar{X}_{k+1} = \bar{X}_k + (t_{k+1} - t_k) \cdot u_k$$

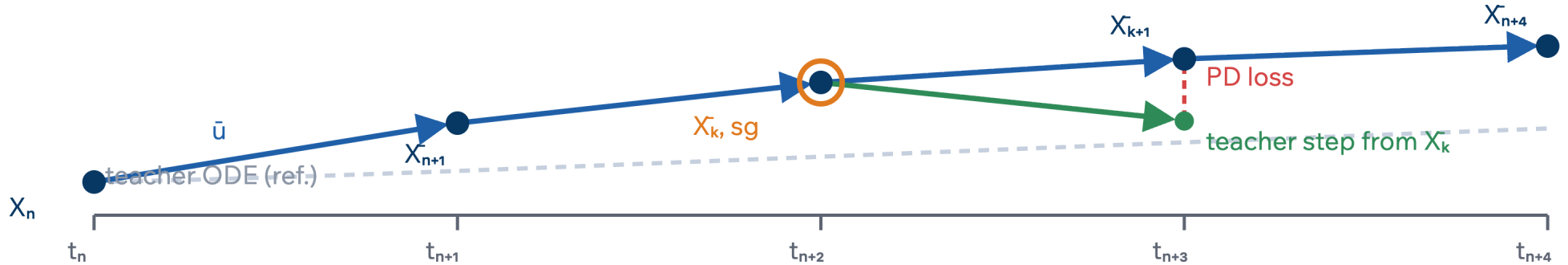
Notation ours. h_θ = shared teacher backbone; W_k = the final linear layer, repeated once per grid interval.

Free at inference. The block's displacement $\Sigma\, \Delta t_k \cdot W_k \cdot h$ is linear in h, so the L heads fuse into one layer: same cost as one teacher call.

Flexible NFE. NFE = N / L. Training with variable block sizes gives one student for 4–8 NFE.

PDD: how the loss is built from a trajectory

One student forward pass at X_n → $\bar{u}_n(n|X_n)$, …, $\bar{u}_n(n+3|X_n)$; roll out $\bar{X}$ with stop-gradient

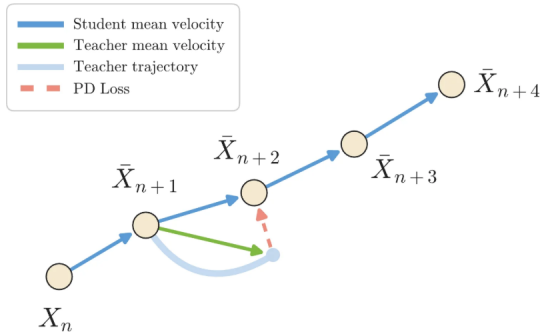


$$L_{PD} = E \| \bar{u}_n(k \mid X_n) - u_k(\, sg(\bar{X}_k)\,) \|^2,\ k \sim U\{n \ldots n+L-1\}$$

$$\bar{X}_{j+1} = \bar{X}_j + h_j \cdot \bar{u}_n(j \mid X_n) \text{ (student rollout)}$$

Euler: $u_k = v(\bar{X}_k, t_k)$

Midpoint: $u_k = v(\bar{X}_k + \tfrac{1}{2}h_k \cdot v(\bar{X}_k, t_k),\ t_k + \tfrac{1}{2}h_k)$



Training PDD: on-policy trajectory matching

1
Start a block at a noisy state X_n

2
One student call; roll out $\bar{X}_{n+1}$ … $\bar{X}_{n+l-1}$ with its own velocities

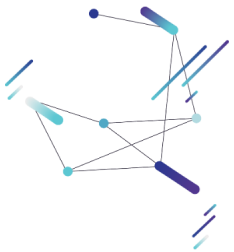
3
Pick one state $\bar{X}_k$ the student reached

4
Teacher step over $[t_k, t_{k+1}]$ at $\bar{X}_k$, stop-grad

5
Regress the student's k-th output onto it: the PD loss

Compared with	What PDD does differently
Progressive distillation	Also regresses on teacher steps, but at the student's own states; no halving rounds
MeanFlow, sCM	Learns mean velocities without JVPs or finite differences
DMD, ADD, LADD	No fake-score network or discriminator; reported better diversity in video

RL analogy: like DAgger, the expert (teacher) labels the learner's own states, removing the train/test mismatch.



PDD results and variants

Wan 2.1 14B text-to-video, 4 NFE (VBench)

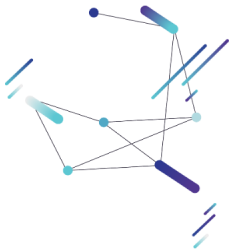
Method	Overall	Quality	Semantic	Diversity ↑
PDD-short	84.92	85.71	81.77	0.0791
DMD2 (FastGen)	84.40	85.16	n/a	0.0568
AnyFlow	84.95	85.70	81.92	n/a

Diversity = mean pairwise V-JEPA 2 cosine distance across 5 videos per prompt. PDD-short: midpoint teacher solver, 200 training iterations.

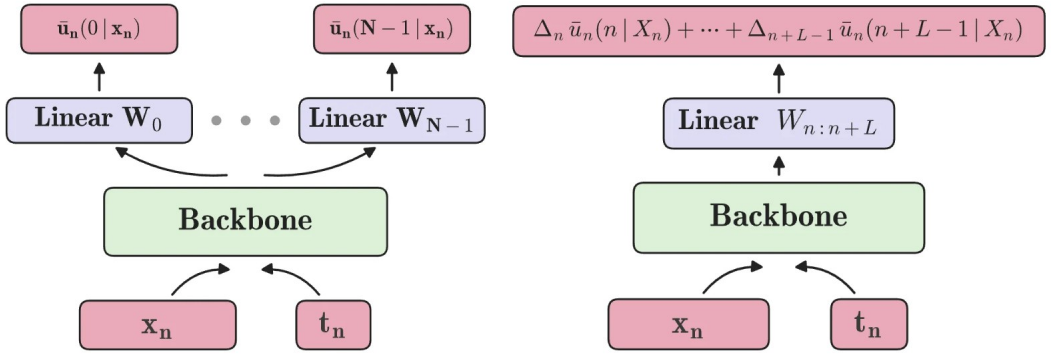
Beats DMD2 on quality and clearly on diversity; on par with AnyFlow on VBench.

Variants and knobs

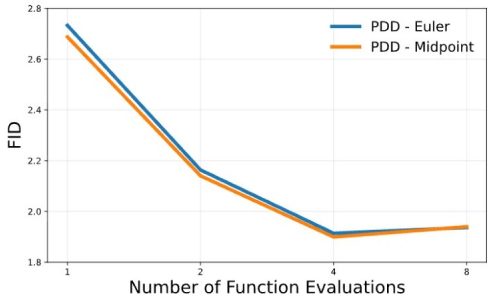
- **Teacher solver** for the target: Euler or midpoint
- **Block size L** sets NFE; variable-L training covers 4–8 NFE
- **PDD-short:** a very short fine-tune of the teacher
- **Models:** LTX-2.3 (text-to-video and audio), Wan 2.1 14B, Qwen-Image
- **Open:** 1–2 NFE, data-dependent settings beyond ImageNet-256; code not yet released



PDD: architecture and training hacks



Train: N heads, one per grid interval, all copies of the teacher's last layer. Generate: a block's heads fuse into one layer, $\Sigma\, h_j W_j$, so 1 NFE costs exactly 1 teacher call.



Hacks

- **Batch size and time-grid shift** matter most (shift s = 5, 6, 10 for Qwen-Image, Wan, LTX).
- **Data-free**: start from noise and roll the student's own trajectory forward with stop-grad.
- **CFG folded** into the teacher velocity; the unconditional branch skips one layer.
- **Grid** N = 256 (Euler) or 128 (midpoint), so both use equal teacher calls.
- **Short**: 250 iterations for Wan / LTX video; ImageNet 1-NFE FID 2.69.

DLS

05

Zero-shot acceleration meets distillation

Fewer tokens, sparser attention, fewer bits, stacked on top of fewer steps

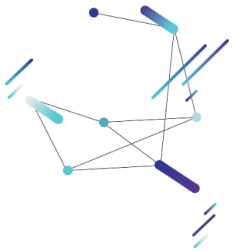
SwD · VSA / Sparse-Distill · TurboDiffusion · quantized few-step students

Steps are only one axis of cost

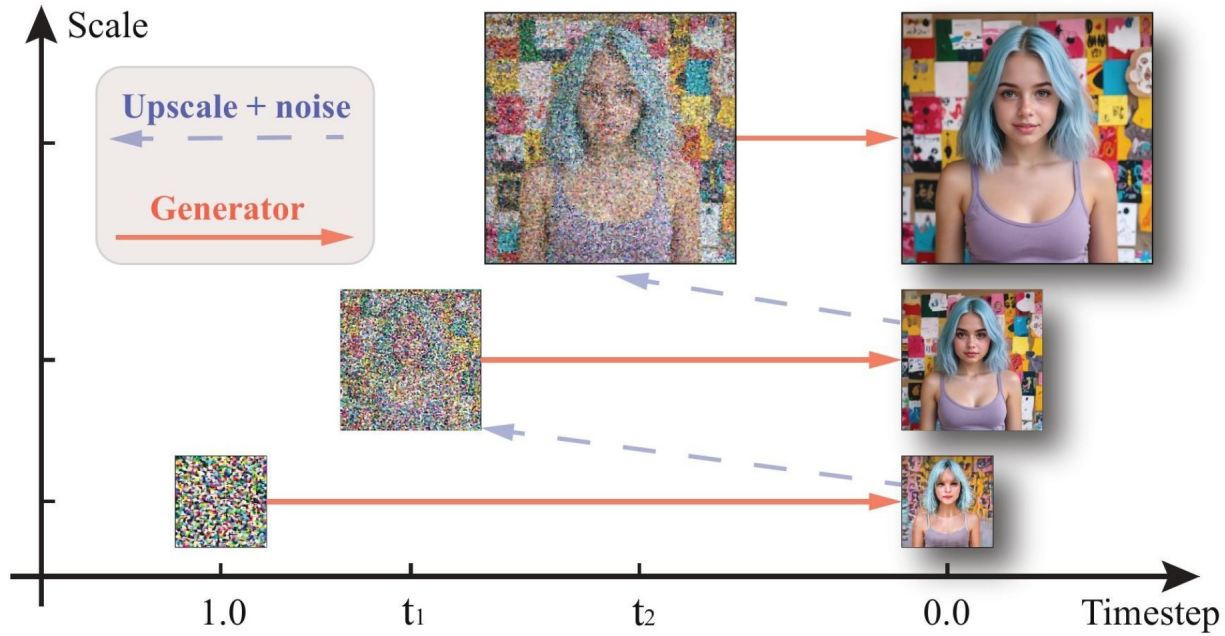
```
cost ≈ steps × calls/step × FLOPs(tokens, attention) × bits
```

Cost axis	Zero-shot trick	Why it breaks after few-step distillation	Co-designed
Resolution	Run noisy steps at low resolution, upscale later	A 4-step student has no “cheap early steps” left to shrink	SwD
Attention	Training-free sparse attention, feature caching	Relies on redundancy across many steps, which distillation removes	VSA + DMD
Precision	PTQ: INT8 weights/activations, INT8 attention	Calibrated on the 50-step trajectory, not on the student’s states	TurboDiffusion, Q-Sched
Steps	DDIM, DPM-Solver	—	Sections 01–04

Lesson: zero-shot tricks exploit redundancy between many steps. Distillation removes that redundancy, so the trick has to be trained or calibrated **together with** the few-step student.



Scale-wise distillation: resolution grows with the steps



Why it works

Noise drowns high frequencies first, so at large t a low-resolution model loses nothing.

Sampling

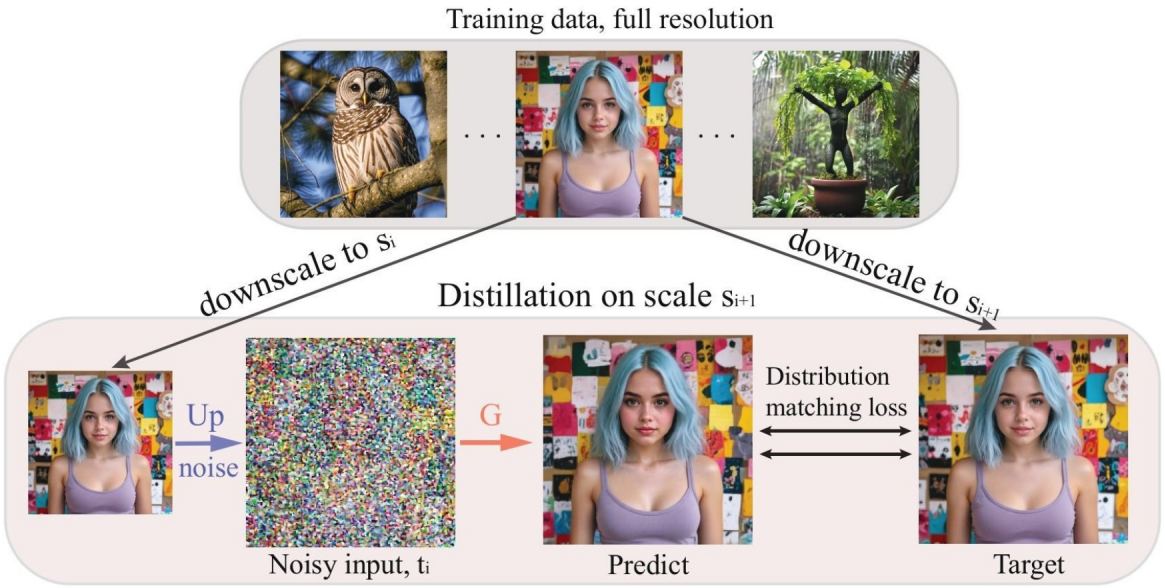
```
t:     1000  895  737  602
scale:  256  512  768 1024
```

$\hat{x}_0$ at s_i → upscale → re-noise to t_{i+1} → step at s_{i+1}

Upscale $\hat{x}_0$, not x_t

FID-5K at t = 600: upscale $\hat{x}_0$, then noise: **18.8**; upscale noisy x_t: **223**.

SwD: training on adjacent scales, and a patch-level MMD loss



Sec / image	SD3.5-M	FLUX	Wan2.1
Full-res, 4 steps	0.29	1.41	5.51
Full-res, 2 steps	0.16	0.72	2.97
Scale-wise, 4 steps	0.17	0.72	1.84

Loss = DMD2 + patch MMD

```
L = L_DMD2 + L_MMD
```

MMD between **token** distributions of student and target, using the teacher's mid-block features, linear kernel. No extra network.

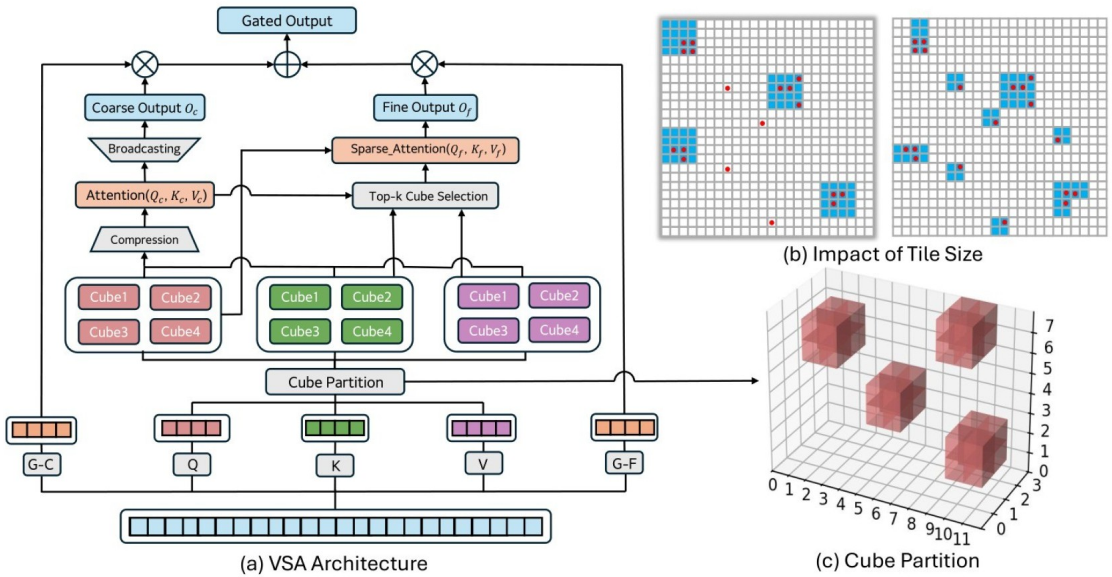
Ablation, MJHQ FID

Full: **13.6** · MMD only: 13.8 · DMD2 without MMD: 19.5 · no time shift: 21.5 · real data instead of teacher data: 28.7

Takeaway

4 scale-wise steps cost about as much as 2 full-resolution steps, and one model is both generator and upscaler.

Sparse-Distill: sparse attention trained together with DMD



(a) VSA Architecture

(b) Impact of Tile Size

(c) Cube Partition

95 s
Wan-1.3B, 50 steps

2.9 s
+ DMD, 3 steps

0.98 s
+ VSA (H200)

VSA: trainable block-sparse attention

Coarse: mean-pool 4×4×4 cubes, attend cube-to-cube, keep top-K cubes per query cube.

Fine: token attention only inside those cubes (block-sparse kernel, ~85% of FA3 MFU).

Output: gated sum of coarse and fine.

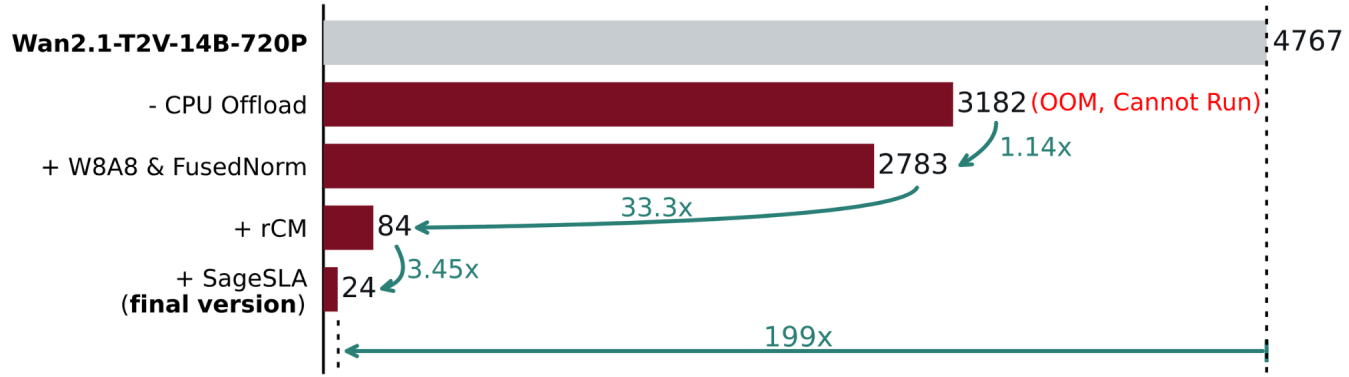
Sparse-Distill

VSA student (0.8 sparsity) + DMD2, 3 steps; dense real score. 4k iters on 64 H200.

Why train jointly

Training-free sparsity reuses patterns of neighbouring steps; 3 steps have none.

TurboDiffusion: stacking quantization, sparsity and rCM



Generation latency (s) on a Single RTX 5090

Four orthogonal tricks

- **SageAttention2++**: INT8 attention
- **SLA**: sparse-linear attn, top-k 10–15%
- **rCM**: 100 → 3–4 steps
- **W8A8**: INT8 linear, 128×128 blocks

How they are combined

From one pretrained model, run **SLA fine-tuning** and **rCM distillation in parallel**, then **merge the weight deltas**. Quantization is post-training.

1.3B-480P: **184 s → 1.9 s**

14B-720P: **4767 s → 24 s**

one RTX 5090

Distillation is the largest factor (33×); the rest multiplies on top.

Quantizing few-step students

Q-Sched: absorb quantization error in the sampler

Post-training quantization of few-step models: LCM, PCM, SDXL-Turbo, FLUX-schnell.

Learn quantization-aware **pre-conditioning coefficients** of the few-step sampler that absorb the quantization error.

JAQ loss: text-image alignment + image quality; a few calibration prompts, no full-precision references.

≈ 4× smaller model; FID 15.5% better than FP16 4-step LCM, 16.6% better than FP16 8-step PCM.

Few-step DMD + low-bit for Wan2.2-A14B

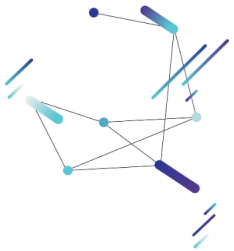
First distill the dual-expert video model with distribution matching, then quantize the **student**.

Calibrate on the **student's own few-step trajectory**, not the teacher's long one: activations differ.

Calibrate high-noise and low-noise experts **separately**; keep sensitive entry layers in high precision; HiF4-style format.

Quantized model stays close to the full-precision model at the same step count.

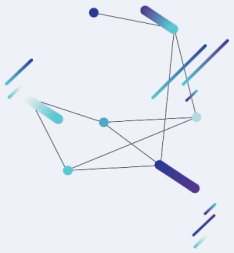
Same lesson as DMD2's backward simulation: whatever you fit, quantizer or student, should see the states the few-step model actually visits at inference.



SUMMARY

The four families side by side

	01 Trajectory	02 Adversarial	03 Distribution	04 PDD
Signal	Teacher ODE or self-consistency	Discriminator	Real − fake score	Teacher step at student states
Extra nets	EMA target	Discriminator heads	Fake score (+ GAN)	None
Real data	Optional	ADD yes, LADD no	Optional	Not needed
Sweet spot	1–4 NFE	1–4 NFE	1–4 NFE	4–8 NFE
Diversity	High	Can drop	Mode-seeking	Reported high
Failure mode	Blur, JVP cost	Instability	Over-saturation	Not yet 1–2 NFE



SUMMARY

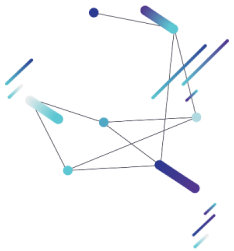
Rules of thumb and open problems

Which one, when?

- One step from scratch: MeanFlow family (iMF, AlphaFlow)
- Big text-to-image model, 1–4 steps: DMD2, LADD, or a hybrid
- Large video, diversity matters, 4–8 steps: PDD or rCM
- Plug-in speed-up for an existing pipeline: LCM-LoRA-style adapters
- Autoregressive video: DMD with self-rollout (Self Forcing)

Open problems

- Sharpness without losing diversity
- Exposure bias: training on the student's own states
- Cheap JVPs at scale, or objectives that avoid them
- Guidance: how much of the gain is just baked-in CFG?
- Evaluation: FID and VBench barely measure diversity



FURTHER READING

References

Trajectory

Salimans & Ho — Progressive Distillation, ICLR 2022

Song et al. — Consistency Models, ICML 2023

Luo et al. — Latent Consistency Models, 2023

Kim et al. — Consistency Trajectory Models, ICLR 2024

Lu & Song — sCM, ICLR 2025 · Zheng et al. — rCM, ICLR 2026

Frans et al. — Shortcut Models, ICLR 2025

Geng et al. — Mean Flows, NeurIPS 2025 · Improved MeanFlow, 2025

AlphaFlow, arXiv 2510.20771

Adversarial

Sauer et al. — ADD, ECCV 2024 · LADD, SIGGRAPH Asia 2024

Lin et al. — SDXL-Lightning, 2024 · Seaweed-APT, 2025

Zero-shot + distillation

Starodubcev et al. — SwD, ICLR 2026

Zhang et al. — VSA, NeurIPS 2025 · TurboDiffusion, 2025

Frumkin & Marculescu — Q-Sched, ICLR 2026

Distribution matching

Yin et al. — DMD, CVPR 2024 · DMD2, NeurIPS 2024

Luo et al. — Diff-Instruct, NeurIPS 2023

Zhou et al. — Score identity Distillation, ICML 2024

Xu, Nie & Vahdat — f-distill, NeurIPS 2024

Yin et al. — CausVid, CVPR 2025 · Huang et al. — Self Forcing, 2025

Liu et al. — Decoupled DMD, ICLR 2026

Luo et al. — TDM, ICCV 2025 · Jiang et al. — DMDR, 2025

Parallel decoding

Shaul et al. — Parallel Decoding Distillation, 2026

Early trajectory

Luhman & Luhman — KD for diffusion, 2021

Liu et al. — Rectified Flow, 2023 · InstaFlow, 2024

Wang et al. — Rectified Diffusion, 2025 · Meng et al., 2023